VECTOR

MICROSAR TCPIP

Technical Reference

Version 14.02.00

Authors	visal, visfdn, viswnk, visjsb, vismjv, visabn, vispcn, vistwe, visbdk, vistbk
Status	Released

Document Information

History

Author	Date	Version	Remarks
visal visfdn	2015-05-18	1.0.0	Creation of document. This document combines the Technical References of the former modules TcpIp, IpV4 and IpV6.
visfdn	2015-10-19	1.0.1	Added Known Issues (IETF RFC Conformance)
visfdn visal	2016-01-13	2.0.0	Added IPv4 Fragmentation Added API to read the local physical address Added sending the FQDN option via DHCP to identify the ECU Added triggering of IP address assignment
visal	2016-03-15	2.1.0	TLS as TcpIp plug-in
visfdn	2016-06-29	3.0.0	Added description of unicast address assignment methods Added description of configuration of static on-link prefixes for IPv6
visfdn	2016-09-22	3.1.0	Added description of optional callout for link-local address assignment.
visfdn	2016-10-20	4.0.0	Updated description of MainFunction, <Up>_CopyTxData() Callback and Tls Heartbeat.
visfdn	2017-01-18	4.1.0	Release of DHCPv4 Server.
viswnk	2017-03-17	5.0.0	Added Production Error Reporting and Other Error Reporting and Diagnostic Options
visjsb viswnk	2017-04-28 2017-05-03	6.0.0	Added description of TcpIp_ClearARCache() Extended Production Error Reporting, Added 5.5.1.7 TcpIpDuplicateAddressDetectionConfig and 3.6.3.4 IPv6 Duplicate Address Detection Conflict
visjsb	2017-05-23	6.1.0	Added description of TcpIp_GetNdpCacheEntries() according ASR 4.3
vismjv	2017-05-30	6.1.0	Added description of TcpIp_GetArpCacheEntries() according ASR 4.3
visal	2017-08-10	6.2.0	Add new socket owner callback for TLS validation result
vismjv	2018-03-15	7.0.0	Added Support of DHCPv4 Requested IP Address Callout
visfdn	2018-05-29	7.0.1	Updated list of “Dynamic Files”.
visfdn	2018-07-04	8.0.0	Updated API documentation. Removed legacy APIs.
visabn	2019-03-05	9.0.0	Updated Component History. Updated Scalability Class reference. Added description for TcpIp_SendGratuitousArpReq () TcpIp_MainFunctionRx (), TcpIp_MainFunctionTx (), TcpIp_MainFunctionState ()
vispcn	2019-03-22	10.00.00	Added DHCP user options and DHCP event callout function.

vistwe	2019-05-02	10.00.01	Added chapter “Security Information”
vispcn	2019-07-16	10.00.02	Improved feature description of DHCP user option handling.
vismjv visabn	2019-08-05 2019-08-12	12.00.00	Added service to read and reset measurement data for diagnostic purpose (TcpIp_GetAndResetMeasurementData() API) Added service to support IPsec Authentication Header for IpV4
vismjv	2019-09-09	13.00.00	Added support of Transmission of Gratuitous ARP reply on startup
visabn	2020-01-28	13.00.01	Extended TcpIp_GetAndResetMeasurementData() to support IPsec SA diagnostics
vismjv	2020-02-25	13.00.02	Extended TcpIp_GetAndResetMeasurementData() to support for insufficient TCP Tx/Rx buffer and IpV4 reassembly buffer
vismjv	2020-02-25	13.00.02	Updated configuration parameter of <Up_PhysAddrTableEntryDiscarded> callout
visbdk	2020-04-29	13.00.03	Update description of TcpIp_RequestComMode() API
vismjv	2020-05-06	14.00.00	Added Support of AUTOSAR parameter TcpIpArpRequestTimeout
vismjv	2020-05-28	14.00.01	Added Support of Icmpv4 Destination Unreachable Message
visbdk	2020-10-12	14.00.02	Names of authors removed. Chapter ‘Component History’ removed. Extend description of TcpIp_Bind API. Add chapter ‘Timing Considerations’.
vispcn	2020-10-27	14.01.00	Updated description of configuration variants and initialization of the module.
vispcn, vismjv	2020-11-13	14.01.01	Added chapter about generation customizations of configuration dependent data. Updated parameter name of API TcpIp_GetDhcpTimeoutInfo().
vistbk	2020-02-18	14.02.00	Added chapter about Static Default Router Configuration. Clarified description of TCPIP_MEAS_DROP_UDP/ TCPIP_MEAS_DROP_TCP. (ESCAN00108259).

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	AUTOSAR_SWS_TcpIp	V4.2.1
[2]	AUTOSAR	AUTOSAR_SWS_DefaultErrorTracer	V4.2.1
[3]	AUTOSAR	AUTOSAR_SWS_DiagnosticEventManager	V4.2.1
[4]	AUTOSAR	AUTOSAR_TR_BSWModuleList	V4.2.1
[5]	AUTOSAR	AUTOSAR_SWS_EthernetInterface	V4.2.1
[6]	AUTOSAR	AUTOSAR_SWS_SocketAdaptor	V4.2.1
[7]	MICROSAR	TechnicalReference_CommonAsr_ComStackLib	see delivery
[8]	IETF RFC768	User Datagram Protocol	Aug 1980
[9]	IETF RFC791	Internet Protocol	Sep 1981
[10]	IETF RFC792	Internet Control Message Protocol (ICMP)	Sep 1981
[11]	IETF RFC793	Transmission Control Protocol	Sep 1981
[12]	IETF RFC813	Window and Acknowledgement Strategy in TCP	Jul 1982
[13]	IETF RFC826	An Ethernet Address Resolution Protocol (ARP)	Nov 1982
[14]	IETF RFC894	A Standard for the Transmission of IP Datagrams over Ethernet Networks	Apr 1984
[15]	IETF RFC1112	Host Extensions for IP Multicasting	Aug 1989
[16]	IETF RFC1122	Requirements for Internet Hosts -- Communication Layers	Oct 1989
[17]	IETF RFC1323	TCP Extensions for High Performance	May 1992
[18]	IETF RFC1981	Path MTU Discovery for IP version 6	Aug 1996
[19]	IETF RFC2018	TCP Selective Acknowledgment Options	Oct 1996
[20]	IETF RFC2131	Dynamic Host Configuration Protocol (DHCP)	Mar 1997
[21]	IETF RFC2460	Internet Protocol, Version 6 (IPv6) Specification	Dec 1998
[22]	IETF RFC2464	Transmission of IPv6 Packets over Ethernet Networks	Dec 1998
[23]	IETF RFC2711	IPv6 Router Alert Option	Oct 1999
[24]	IETF RFC3122	Extensions to IPv6 Neighbor Discovery for Inverse Discovery Specification	Jun 2001
[25]	IETF RFC3315	Dynamic Host Configuration Protocol for IPv6 (DHCPv6)	Jul 2003
[26]	IETF RFC3810	Multicast Listener Discovery Version 2 (MLDv2) for IPv6	Jun 2004
[27]	IETF RFC3927	Dynamic Configuration of IPv4 Link-Local Addresses	May 2005
[28]	IETF RFC4291	IP Version 6 Addressing Architecture	Feb 2006
[29]	IETF RFC4429	Optimistic Duplicate Address Detection (DAD) for IPv6	Apr 2006
[30]	IETF RFC4443	Internet Control Message Protocol (ICMPv6) for the IPv6 Specification	Mar 2006
[31]	IETF RFC4702	The DHCP Client FQDN option	Oct 2006
[32]	IETF RFC4704	The DHCPv6 Client FQDN option	Oct 2006
[33]	IETF RFC4861	Neighbor Discovery for IP version 6 (IPv6)	Sep 2007
[34]	IETF RFC4862	IPv6 Stateless Address Autoconfiguration	Sep 2007
[35]	IETF RFC4884	Extended ICMP to Support Multi-Part Messages	Apr 2007

[36]	IETF RFC4941	Privacy Extensions for Stateless Address Autoconfiguration in IPv6	Sep 2007
[37]	IETF RFC5095	Deprecation of Type 0 Routing Headers in IPv6	Dec 2007
[38]	IETF RFC5482	TCP User Timeout Option	Mar 2009
[39]	IETF RFC5681	TCP Congestion Control	Sep 2009
[40]	IETF RFC5722	Handling of Overlapping IPv6 Fragments	Dec 2009
[41]	IETF RFC5942	IPv6 Subnet Model: The Relationship between Links and Subnet Prefixes	Jul 2010
[42]	IETF RFC6085	Address Mapping of IPv6 Multicast Packets on Ethernet	Jan 2011
[43]	IETF RFC6106	IPv6 Router Advertisement Options for DNS Configuration	Nov 2010
[44]	IETF RFC6298	Computing TCP's Retransmission Timer	Jun 2011
[45]	IETF RFC6582	The NewReno Modification to TCP's Fast Recovery Algorithm	Apr 2012
[46]	IETF RFC6724	Default Address Selection for Internet Protocol version 6 (IPv6)	Sep 2012
[47]	IETF RFC4301	Security Architecture for the Internet Protocol	Dec 2005
[48]	IETF RFC4302	IP Authentication Header	Dec 2005
[49]	IETF RFC4543	The Use of Galois Message Authentication Code (GMAC) in IPsec ESP and AH	May 2006
[50]	IANA-IKEv2	Internet Key Exchange Version 2 (IKEv2) Parameters	May 2019
[51]	Vector	TechnicalReference_Tls.pdf	V4.00.00

Scope of the Document

This technical reference describes the general use of the TCPIP basis software. Please refer to your Release Notes to get a detailed description of the platform (host, compiler, controller) your Vector Ethernet Bundle has been configured for.



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

Contents

1 Introduction 14
1.1 Architecture Overview 16

2 Functional Description 17
2.1 Features 17
2.1.1 IETF RFC Conformance 18
2.1.2 Additions / Extensions 20
2.1.3 Limitations 22
2.1.4 Known Issues (low priority) 26
2.1.5 Known Issues (IETF RFC Conformance) 26
2.2 Structure of the TcpIp Stack 28
2.2.1 IpV6 28
2.2.2 IpV4 29
2.2.3 TLS 29
2.2.4 DHCPv4 Server 29
2.3 Initialization 31
2.4 States 31
2.5 Main Functions 32
2.6 Timing Considerations 32
2.7 Error Handling 33
2.7.1 Development Error Reporting 33
2.7.2 Production Code Error Reporting 37
2.7.3 Other Error Reporting and Diagnostic Options 38

3 Integration 40
3.1 Scope of Delivery 40
3.1.1 Static Files 40
3.1.2 Dynamic Files 41
3.2 Critical Sections 42
3.3 Preemption 42

4 API Description 43
4.1 Type Definitions 43
4.2 Services provided by TCPIP 54
4.2.1 TcpIp_GetVersionInfo 54
4.2.2 TcpIp_InitMemory 55
4.2.3 TcpIp_Init 56
4.2.4 TcpIp_<Up>GetSocket 57
4.2.5 TcpIp_GetSocketForUser 58

4.2.6 TcpIp_ChangeParameter 59
4.2.7 TcpIp_Bind 60
4.2.8 TcpIp_UdpTransmit 61
4.2.9 TcpIp_TcpListen 62
4.2.10 TcpIp_TcpConnect 63
4.2.11 TcpIp_TcpTransmit 64
4.2.12 TcpIp_TcpReceived 65
4.2.13 TcpIp_Close 66
4.2.14 TcpIp_IcmpV6Transmit 67
4.2.15 TcpIp_RequestIpAddrAssignment 68
4.2.16 TcpIp_ReleaseIpAddrAssignment 69
4.2.17 TcpIp_ReleaseSpecificIpAddrAssignment 69
4.2.18 TcpIp_GetCtrlIdx 70
4.2.19 TcpIp_GetIpAddr 71
4.2.20 TcpIp_GetIpAddrCfgSrc 72
4.2.21 TcpIp_GetRemNetAddr 72
4.2.22 TcpIp_GetLocSockAddr 73
4.2.23 TcpIp_GetPhysAddr 73
4.2.24 TcpIp_GetRemotePhysAddr 74
4.2.25 TcpIp_Tls_SetClientCertInfo 75
4.2.26 TcpIp_Tls_GetNvmBlockIdForUsedRootCert 75
4.2.27 TcpIp_Tls_RootCertWasModified 76
4.2.28 TcpIp_DhcpReadOption 76
4.2.29 TcpIp_DhcpWriteOption 77
4.2.30 TcpIp_DhcpV6ReadOption 78
4.2.31 TcpIp_DhcpV6WriteOption 79
4.2.32 TcpIp_GetDhcpTimeoutInfo 80
4.2.33 TcpIp_DhcpV4_GetStatus 81
4.2.34 TcpIp_RequestComMode 82
4.2.35 TcpIp_DiagDataReadAccess 83
4.2.36 TcpIp_ClearARCache 83
4.2.37 TcpIp_GetArpCacheEntries 84
4.2.38 TcpIp_GetNdpCacheEntries 85
4.2.39 TcpIp_SendGratuitousArpReq 86
4.2.40 IpV4_GetLastDuplicateDhcpAddrDid 87
4.2.41 TcpIp_MainFunction 87
4.2.42 TcpIp_MainFunctionRx 88
4.2.43 TcpIp_MainFunctionTx 88
4.2.44 TcpIp_MainFunctionState 89
4.2.45 TcpIp_GetLocNetAddr 89
4.2.46 TcpIp_GetLocNetMask 90

4.2.47 TcpIp_GetAndResetMeasurementData 91
4.2.48 TcpIp_IpSecSaEntryPairAdd 92
4.2.49 TcpIp_IpSecSaEntryPairDelete 93
4.3 Services used by TCPIP 93
4.4 Callback Functions 93
4.4.1 TcpIp_RxIndication 94
4.5 Configurable Interfaces 95
4.5.1 Notifications and Callouts 95

5 Configuration 107
5.1 Configuration Variants 107
5.2 Socket Owner Configuration 107
5.2.1 <Up>_CopyTxData Callback 108
5.3 Unicast Address Assignment Methods 108
5.3.1 Multiple Address Assignment Methods for IPv4 Unicast Addresses 110
5.3.2 Callout for provision of IPv4 link-local address candidates 111
5.3.3 Callout for provision of DHCPv4 address that shall be requested 111
5.4 Configuration of Static On-Link Prefixes for IPv6 112
5.5 Static and Dynamic Link Layer Address Resolution 112
5.5.1 Prevention of ARP Flooding 113
5.6 DHCP user options 113
5.7 IPsec Configuration 114
5.7.1 SPD Configuration 114
5.7.2 Integrity Algorithms 115
5.7.3 Encryption Algorithms 115
5.8 Static Default Router Configuration 115
5.9 Generation customizations 115

6 Security Information 116
6.1 ARP 116
6.2 ICMPv4 117
6.3 IPv4 117
6.4 IPv6 117
6.5 TCP 118
6.6 UDP 119
6.7 DHCPv4 Server 119

7 Glossary and Abbreviations 120
7.1 Glossary 120
7.2 Abbreviations 120

8 Contact 121

Illustrations

Figure 1-1 AUTOSAR 4.2 Architecture Overview 16

Tables

Table 2-1 Supported AUTOSAR R4.2.1 standard conform features 17
Table 2-2 Supported AUTOSAR R4.3.0 standard conform features 17
Table 2-3 Not supported AUTOSAR R4.2.1 standard conform features 18
Table 2-4 List of supported TcpIp IETF RFCs 18
Table 2-5 List of supported IPv4 IETF RFCs 19
Table 2-6 List of supported IPv6 IETF RFCs 19
Table 2-7 Features provided beyond the AUTOSAR standard 20
Table 2-8 Deviations from IETF RFC4301 23
Table 2-9 Deviations from IETF RFC4302 23
Table 2-10 Deviations from IETF RFC4301 23
Table 2-11 Deviations from IETF RFC791 23
Table 2-12 Deviations from IETF RFC792 23
Table 2-13 Deviations from IETF RFC1323 24
Table 2-14 Deviations from IETF RFC2131 24
Table 2-15 Deviations from IETF RFC4702 24
Table 2-16 Deviations from IETF RFC2460 24
Table 2-17 Deviations from IETF RFC3315 24
Table 2-18 Deviations from IETF RFC3484 25
Table 2-19 Deviations from IETF RFC4291 25
Table 2-20 Deviations from IETF RFC4443 25
Table 2-21 Deviations from IETF RFC4443 25
Table 2-22 MainFunctions of the TcpIp module 32
Table 2-23 Service IDs 36
Table 2-24 Errors reported to DET 37
Table 2-25 Errors reported to DEM 38
Table 3-1 Static files 41
Table 3-2 Generated files 41
Table 4-1 Type definitions 51
Table 4-2 TcpIp_SockAddrType 51
Table 4-3 TcpIp_SockAddrInetType 52
Table 4-4 TcpIp_SockAddrInet6Type 53
Table 4-5 TcpIp_IpSecKeyMaterialInfoType 53
Table 4-6 TcpIp_IpSecTransformInfoType 53
Table 4-7 TcpIp_IpSecTrafficSelectorType 54
Table 4-8 TcpIp_IpSecSaInfoType 54
Table 4-9 TcpIp_IpSecSaStatusType 54
Table 4-10 TcpIp_GetVersionInfo 55
Table 4-11 TcpIp_InitMemory 55
Table 4-12 TcpIp_Init 56
Table 4-13 TcpIp_<Up>GetSocket 57
Table 4-14 TcpIp_GetSocketForUser 58
Table 4-15 TcpIp_ChangeParameter 59
Table 4-16 TcpIp_Bind 60
Table 4-17 TcpIp_UdpTransmit 61
Table 4-18 TcpIp_TcpListen 62

Table 4-19 TcpIp_TcpConnect 63
Table 4-20 TcpIp_TcpTransmit 64
Table 4-21 TcpIp_TcpReceived 65
Table 4-22 TcpIp_Close 66
Table 4-23 TcpIp_IcmpV6Transmit 67
Table 4-24 TcpIp_RequestIpAddrAssignment 68
Table 4-25 TcpIp_ReleaseIpAddrAssignment 69
Table 4-26 TcpIp_ReleaseIpAddrAssignment 69
Table 4-27 TcpIp_GetCtrlIdx 70
Table 4-28 TcpIp_GetIpAddr 71
Table 4-29 TcpIp_GetIpAddrCfgSrc 72
Table 4-30 TcpIp_GetRemNetAddr 72
Table 4-31 TcpIp_GetLocSockAddr 73
Table 4-32 TcpIp_GetPhysAddr 73
Table 4-33 TcpIp_GetRemotePhysAddr 74
Table 4-34 TcpIp_Tls_SetClientCertInfo 75
Table 4-35 TcpIp_Tls_GetNvmBlockIdForUsedRootCert 75
Table 4-36 TcpIp_Tls_RootCertWasModified 76
Table 4-37 TcpIp_DhcpReadOption 76
Table 4-38 TcpIp_DhcpWriteOption 77
Table 4-39 TcpIp_DhcpV6ReadOption 78
Table 4-40 TcpIp_DhcpV6WriteOption 79
Table 4-41 TcpIp_GetDhcpTimeoutInfo 80
Table 4-42 TcpIp_DhcpV4_GetStatus 81
Table 4-43 TcpIp_RequestComMode 82
Table 4-44 TcpIp_DiagDataReadAccess 83
Table 4-45 TcpIp_ClearARCache 83
Table 4-46 TcpIp_GetArpCacheEntries 84
Table 4-47 TcpIp_GetNdpCacheEntries 85
Table 4-48 TcpIp _SendGratuitousArpReq 86
Table 4-49 IpV4_GetLastDuplicateDhcpAddrDid 87
Table 4-50 TcpIp_MainFunction 87
Table 4-51 TcpIp_MainFunctionRx 88
Table 4-52 TcpIp_MainFunctionTx 88
Table 4-53 TcpIp_MainFunctionState 89
Table 4-54 TcpIp_GetLocNetAddr 89
Table 4-55 TcpIp_GetLocNetMask 90
Table 4-56 TcpIp_GetAndResetMeasurementData 91
Table 4-57 TcpIp_IpSecSaEntryPairAdd 92
Table 4-58 TcpIp_IpSecSaEntryPairDelete 93
Table 4-59 TcpIp_RxIndication 94
Table 4-60 Maximum Payload Length Change Callback 95
Table 4-61 Inverse Neighbor Discovery Address List Option Receive Callback 96
Table 4-62 ICMP Destination Unreachable Message Callback 97
Table 4-63 ICMPV6 Destination Unreachable Message Callback 98
Table 4-64 IPv4 Link local Auto Ip address assignment Callback 99
Table 4-65 Physical Address Change Callback 100
Table 4-66 Physical Address Change Discarded Callback 101
Table 4-67 Duplicate Address Detection Callback 102
Table 4-68 DHCPv4 Requested IP address 103
Table 4-69 DHCP Event Callout 104
Table 4-70 IPsec Security Policy Callout 105
Table 4-71 IPsec Audit Event Callout 106
Table 5-1 Callbacks of TcpIp Socket Ownerss 107

Table 5-2 Configuration possibilities for IPv4 and IPv6 unicast address assignments 108
Table 6-1 TcpIpArpEnabled, ARP spoofing 116
Table 6-2 TcpIpArpEnabled, ARP flooding 116
Table 6-3 TcpIpArpDiscardedEntryHandling 117
Table 6-4 TcpIpIcmpEnabled 117
Table 6-5 TcpIpIpV4Enabled 117
Table 6-6 TcpIpIpV6Enabled 118
Table 6-7 TcpIpTcpEnabled, port scan 118
Table 6-8 TcpIpTcpEnabled, man in the middle 118
Table 6-9 TcpIpUdpEnabled 119
Table 6-10 TcpIpDhcpServerEnabled 119
Table 7-1 Glossary 120
Table 7-2 Abbreviations 120

1 Introduction

This document describes the functionality, API and configuration of the AUTOSAR BSW module TCPIP as specified in [1].

Supported AUTOSAR Release*:	4	
Supported Configuration Variants:	pre-compile	
Vendor ID:	TCPIP_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	TCPIP_MODULE_ID	170 decimal (according to ref. [4])

* For the detailed functional specification please also refer to the corresponding AUTOSAR SWS.

The TcpIp provides access to a socket interface based on the Internet Protocol (IPv4 and IPv6) over Ethernet.

The TcpIp module consists of 5 submodules:

- TcpIp (TCP, UDP, Socket interface)
- IpV4 (IPv4, ARP, ICMPv4, DHCPv4 client)
- IpV6 (IPv6, NDP, ICMPv6, DHCPv6 client)
- DhcpV4Server (DHCPv4 server)
- IpSec (Authentication Header for IpV4)



Note
The submodules **IpV4**, **IpV6, IpSec** and **DhcpV4Server** must be licensed separately for use in production code. Use of all the submodules is allowed in development code.

Scalability Classes (SC*) based on different combinations of IP submodules are possible to be configured in the TCPIP:

Scalability Class	Submodules
SC 1	**IpV4** only
SC 2	**IpV6** only
SC 3	**IpV4**/**IpV6** Dual stack

TcpIp submodule is configured by default in all SC.

DhcpV4Server submodule may be configured only in SC1 and SC3.
IpSec submodule may be configured only in SC1 and SC3.

* SC: Scalability Class according to [1] AUTOSAR_SWS_TcpIp

1.1 Architecture Overview

The following figure shows where the TCPIP is located in the AUTOSAR architecture.

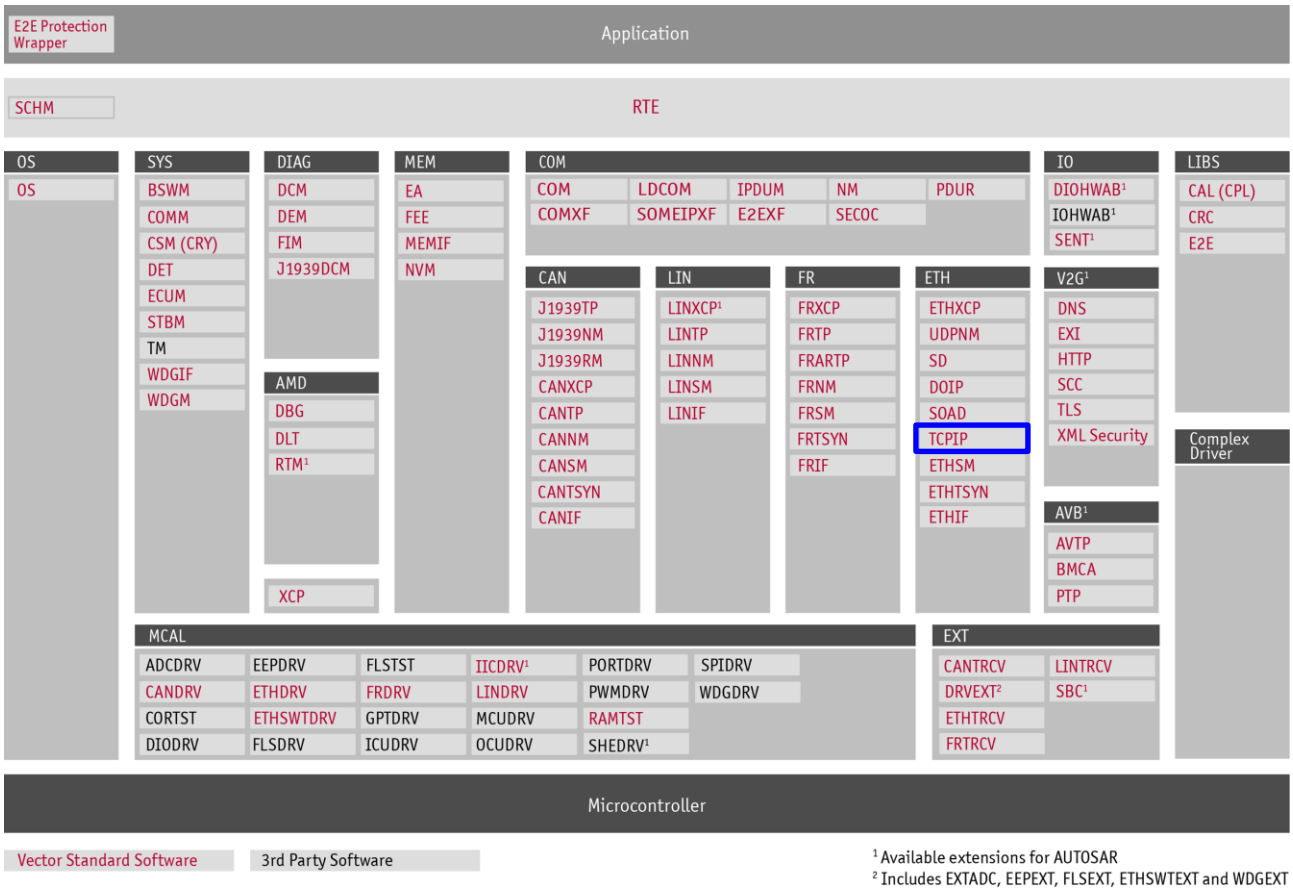


Figure 1-1 AUTOSAR 4.2 Architecture Overview

2 Functional Description

2.1 Features

The features listed in the following tables cover the complete functionality specified for the TCPIP.

The AUTOSAR standard functionality is specified in [1], the corresponding features are listed in the tables

> Table 2-1 Supported AUTOSAR R4.2.1 standard conform features

> Table 2-2 Supported AUTOSAR R4.3.0 standard conform features

> Table 2-3 Not supported AUTOSAR R4.2.1 standard conform features

Vector Informatik provides further TCPIP functionality beyond the AUTOSAR standard. The corresponding features are listed in the table

> Table 2-7 Features provided beyond the AUTOSAR standard

Supported AUTOSAR R4.2.1 Standard Conform Features
Configuration according to AUTOSTAR R4.2.1
Implementation of TCP and UDP
Implementation of IPv4 (incl. ARP, ICMPv4, DHCPv4 client/ server) with multi controller support
Implementation of IPv6 (incl. NDP, ICMPv6, DHCPv6 client) with multi controller support

Table 2-1 Supported AUTOSAR R4.2.1 standard conform features

Supported AUTOSAR R4.3.0 Standard Conform Features
Support for read and reset of diagnostic measurement data

Table 2-2 Supported AUTOSAR R4.3.0 standard conform features

Not supported AUTOSAR R4.2.1 standard conform features	
TcpIp	Production error reporting.
TcpIp	TCP/IP Stack State handling with state ONHOLD
TcpIp	API TcpIp_RequestComMode (empty API implementation only)
TcpIp	API TcpIp_ResetIpAssignment
TcpIp / TCP	API TcpIp_TcpTransmit: ForceRetrieve parameter always assumed as TRUE.
TcpIp / TCP	Simultaneous Open Attempts, Delayed ACKs.
TcpIp / TCP	The following configuration parameters are currently not used: TcpIpTcpReceiveWindowMax, TcpIpTcpMaxRtx, TcpIpTcpSynMaxRtx, TcpIpTcpSynReceivedTimeout, TcpIpTcpFinWait2Timeout
TcpIp / UDP	Forward received multicasts to all matching sockets.
IpV4	Path MTU Discovery for IPv4 (IETF RFC1191).
IpV4 / ARP	Extract physical address information from each received IP packet.

Not supported AUTOSAR R4.2.1 standard conform features	
IpV4 / ICMP	Transmit ICMP messages other than Echo Request / Echo Reply and Destination Unreachable message. Receive ICMP messages other than Echo Request and Echo Reply. Configuration of ICMP message handler.
IpV4 / ICMP	API TcpIp_IcmpTransmit

Table 2-3 Not supported AUTOSAR R4.2.1 standard conform features

2.1.1 IETF RFC Conformance

The TcpIp module conforms to the following RFCs published by the Internet Engineering Task Force (IETF) with the noted exceptions.

> RFC referenced by the AUTOSAR SWS TcpIp R4.2.1
> RFC supported beyond the AUTOSAR standard

Note
Unless otherwise noted the following parts of the IETF RFCs are not implemented:
> Sections that specify server/router specific functionality
> Sections that are excluded by the AUTOSAR specification

IETF RFC	Title
768	User Datagram Protocol
793	Transmission Control Protocol
813	Window and Acknowledgement Strategy in TCP
896	Congestion Control in IP/TCP Internetworks (Nagle)
1122	Requirements for Internet Hosts -- Communication Layers
1323	TCP Extensions for High Performance
2018	TCP Selective Acknowledgment Options
3390	Increasing TCP's Initial Window
5482	TCP User Timeout Option
5681	TCP Congestion Control
6298	Computing TCP's Retransmission Timer
6582	The NewReno Modification to TCP's Fast Recovery Algorithm
4301	Security Architecture for the Internet Protocol
4302	IP Authentication Header
4543	The Use of Galois Message Authentication Code (GMAC) in IPsec ESP and AH

Table 2-4 List of supported TcpIp IETF RFCs

IETF RFC	Title
791	Internet Protocol
826	An Ethernet Address Resolution Protocol (ARP)
792	Internet Control Message Protocol (ICMP)
815	IP Datagram Reassembly Algorithms
894	A Standard for the Transmission of IP Datagrams over Ethernet Networks
1112	Host Extensions for IP Multicasting
2131	Dynamic Host Configuration Protocol (DHCP) / client and server
3925	Vendor-Identifying Vendor Options for Dynamic Host Configuration Protocol version 4 (DHCPv4)
3927	Dynamic Configuration of IPv4 Link-Local Addresses

Table 2-5 List of supported IPv4 IETF RFCs

IETF RFC	Title
1981	Path MTU Discovery for IP version 6
2460	Internet Protocol, Version 6 (IPv6) Specification
2464	Transmission of IPv6 Packets over Ethernet Networks
2711	IPv6 Router Alert Option
3122	Extensions to IPv6 Neighbor Discovery for Inverse Discovery Specification
3315	Dynamic Host Configuration Protocol for IPv6 (DHCPv6) / client
3810	Multicast Listener Discovery Version 2 (MLDv2) for IPv6
4291	IP Version 6 Addressing Architecture
4429	Optimistic Duplicate Address Detection (DAD) for IPv6
4443	Internet Control Message Protocol (ICMPv6) for the IPv6 Specification
4861	Neighbor Discovery for IP version 6 (IPv6)
4862	IPv6 Stateless Address Autoconfiguration
4884	Extended ICMP to Support Multi-Part Messages
4941	Privacy Extensions for Stateless Address Autoconfiguration in IPv6
5095	Deprecation of Type 0 Routing Headers in IPv6
5722	Handling of Overlapping IPv6 Fragments
5942	IPv6 Subnet Model: The Relationship between Links and Subnet Prefixes
6085	Address Mapping of IPv6 Multicast Packets on Ethernet
6106	IPv6 Router Advertisement Options for DNS Configuration
6724	Default Address Selection for Internet Protocol version 6 (IPv6)

Table 2-6 List of supported IPv6 IETF RFCs

2.1.2 Additions / Extensions

The following features are provided beyond the AUTOSAR standard:

Features Provided Beyond the AUTOSAR Standard	
API TcpIp_GetLocNetAddr: Get the local IPv4 address	4.2.45
API TcpIp_GetLocNetMask: Get the local IPv4 network mask	4.2.46
API TcpIp_GetIpAddrCfgSrc: Get the address configuration source of a local address	4.2.33
API TcpIp_DiagDataReadAccess: Read access to some internal variables	4.2.35
API TcpIp_GetDhcpTimeoutInfo: Get the information if a DHCP timeout has occurred	4.2.32
API TcpIp_GetLocSockAddr: Get the current local socket address for one socket	4.2.22
API TcpIp_GetRemNetAddr: Get the current remote socket address for one socket	4.2.20
IPv4 Manual override of any IPv4 local unicast address during runtime	5.2.1
IpV4 Configuration of static ARP Tables (disable dynamic address resolution)	5.3.2
IpV4: Static Link Layer Address Resolution	5.5
IpV4/IpV6: Persistence of static IPv4/IPv6 local unicast address	5.2.1
IpV6: Transmit and reassemble fragmented packets	5.2.1
IpV6: Inverse Neighbor Discovery	
TcpIp / TCP Reception of out-of-order segments	2.1.2.1.1
TcpIp / TCP Selective Acknowledgement	2.1.2.1.2
TcpIp / TCP: Use TLS as a TCP plug-in to encrypt and authenticate a TCP connection	4.2.6 4.2.25 4.2.26 4.2.27
TcpIp: Separate MainFunctions for reception/transmission of user data and state handling.	2.5
TcpIp: Extended <Up>_CopyTxData-Callback for transmission of data with dynamic length.	5.2.1
API TcpIp_ClearARCache: Clear the address resolution cache.	4.2.36
DHCP event callout which is invoked for each received and prior transmission of client DHCP message.	4.5.1.10
API TcpIp_IpSecSaEntryPairAdd: Insert a pair of SA entries in the Security Association Database	4.2.48
API TcpIp_IpSecSaEntryPairDelete: Delete a pair of SA entries from the Security Association Database	4.2.49

Table 2-7 Features provided beyond the AUTOSAR standard

2.1.2.1 TCP Extensions

2.1.2.1.1 TCP Out Of Order Support

For TCP a support for messages received out of order (OOO) can be enabled. If this feature is enabled TCP stores incoming segments even if previous messages are missing. As soon as the missing segments are received the TCP forwards all stored data to the socket owner.

2.1.2.1.2 TCP Selective Acknowledgement Support

In the standard configuration TCP won't notify the communication partner that it has a gap in its received data sequence. This feature called 'selective acknowledgement' (SACK) can be enabled separately (in addition to OOO). Before SACK can be used it has to be agreed during TCP connection setup (using TCP header options).

2.1.2.2 Parameter Checking

2.1.2.2.1 Parameter checking for submodule TcpIp

The following development errors and error codes are reported additionally to the errors defined in [1].

Development errors:

- TCPIP_API_ID_SHUTDOWN
- TCPIP_API_ID_PROVIDE_TX_BUFFER
- TCPIP_API_ID_TRANSMIT_TO
- TCPIP_API_ID_GET_LOC_NET_ADDR
- TCPIP_API_ID_GET_LOC_NET_MASK
- TCPIP_API_ID_SET_DHCP_HOST_NAME
- TCPIP_API_ID_SET_DHCP_V4_TX_OPT
- TCPIP_API_ID_SET_DHCP_V6_TX_OPT
- TCPIP_API_ID_CBK_RX_INDICATION
- TCPIP_API_ID_CBK_TX_CONFIRMATION
- TCPIP_API_ID_CBK_TCP_ACCEPTED
- TCPIP_API_ID_CBK_TCP_CONNECTED
- TCPIP_API_ID_CBK_TCP_IP_EVENT
- TCPIP_API_ID_RX_SOCK_IDENT
- TCPIP_API_ID_TCP_STATE_CHG
- TCPIP_API_ID_LOC_IP_ASS_CHG
- TCPIP_API_ID_PREPARE_SHUTDOWN
- TCPIP_API_ID_TRCV_LINK_STATE_CHG
- TCPIP_API_ID_ADDR_RES_TIMEOUT

- TCPIP_API_ID_SOCK_ADDR_IS_EQ
- TCPIP_API_ID_SOCK_ADDR_IS_EQ_OR_UNSPEC
- TCPIP_API_ID_PATH_MTU_CHG
- TCPIP_API_ID_GET_IP_ADDR_CFG_SRC
- TCPIP_API_ID_GET_IP_ADDR
- TCPIP_API_ID_GET_REM_NET_ADDR
- TCPIP_API_ID_CLEAR_AR_CACHE
- TCPIP_API_ID_SEND_GRATUITOUS_ARP_REQ

Error codes:

- TCPIP_E_INV_SOCK_ID
- TCPIP_API_ID_GET_AND_RESET_MEAS_DATA
- TCPIP_API_ID_IPSEC_ADD_SAENTRY_PAIR
- TCPIP_API_ID_IPSEC_DEL_SAENTRY_PAIR
- TCPIP_API_ID_IPSEC_SPD_CALLOUT

2.1.3 Limitations

2.1.3.1 General Limitations

- The number of sockets is limited to 254.

2.1.3.2 Limitations for AUTOSAR APIs

2.1.3.2.1 DHCP write and read option

The APIs to write (TcpIp_DhcpWriteOption and TcpIp_DhcpV6WriteOption) as well as to read DHCP options (TcpIp_DhcpReadOption and TcpIp_DhcpV6ReadOption) can only be used to write and read statically configured options. It is not possible to read out or write “unknown” options during runtime.

The APIs are limited to:

- Fully Qualified Domain Name (FQDN) options.
- Statically configured DHCP user options.

2.1.3.2.2 Triggering IP address request

The request of IP address assignment is configurable, but only a configuration subset is supported. All address assignments for one IpAddrId need to have the same configuration.

2.1.3.3 RFC specific deviations of submodule TcpIp

2.1.3.3.1 RFC4301 Security Architecture for the Internet Protocol

Section in Document	Limitation
4.4.1.1. Selectors	ICMP is not supported as a selector

Table 2-8 Deviations from IETF RFC4301

2.1.3.3.2 RFC4302 IP Authentication Header

Section in Document	Limitation
3. Authentication Header Processing	Feature is only supported in combination with IPv4 controllers.
3.1.2 Tunnel Mode	Tunnel mode is not supported.
3.3.4 Fragmentation	Authentication of fragmentated packet is not supported.
4.4.1 The Security Policy Database (SPD)	An additional Policy OPTIONAL is introduced.

Table 2-9 Deviations from IETF RFC4302

2.1.3.3.3 RFC4543 The Use of GMAC in IPsec ESP and AH see [49]

Section in Document	Limitation
All Sections related to ESP	ESP Header is not supported

Table 2-10 Deviations from IETF RFC4301

2.1.3.4 RFC specific deviations of submodule IpV4

2.1.3.4.1 RFC791 Internet Protocol [9]

Section in Document	Limitation
Page 15 / IPv4 Header Options	IP options may appear in received packets but will be silently ignored.

Table 2-11 Deviations from IETF RFC791

2.1.3.4.2 RFC792 Internet Control Message Protocol (ICMP) [10]

Section in Document	Limitation
ICMP Messages	ICMP message support is limited to Echo Request, Echo Reply and transmission of Destination Unreachable messages. The length of the data mirrored in the Echo Reply message is limited (configurable).

Table 2-12 Deviations from IETF RFC792

2.1.3.4.3 RFC1323 TCP Extensions for High Performance [17]

Section in Document	Limitation
2. “TCP Window Scale Option”	Not supported.
4. “PAWS -- Protect Against Wrapped Sequence Numbers”	Not supported.

Table 2-13 Deviations from IETF RFC1323

2.1.3.4.4 RFC2131 Dynamic Host Configuration Protocol (DHCP) [20]

Section in Document	Limitation
General	Basic DHCP client and server support implemented.
3.2 Client-server interaction	“Client-server interaction - reusing a previously allocated network address” is not implemented. DHCPv4 client does not skip sending of DISCOVER messages but it is possible to request a particular IP address by using the <User>_RequestedDhcpAddrCallout.
Probing of Addresses	IP addresses assigned by DHCP are not probed by the IP-Module to be unique.
4.3.5 “DHCPINFORM message”	DHCPINFORM messages are not send by the client and are ignored by the DHCP server.

Table 2-14 Deviations from IETF RFC2131

2.1.3.4.5 RFC4702 The DHCP Client FQDN option [31]

Section in Document	Limitation
General	The ‚Flags‘ field is filled with fixed values. No DNS update handling is implemented in any way.

Table 2-15 Deviations from IETF RFC4702

2.1.3.5 RFC specific deviations of submodule IpV6

2.1.3.5.1 RFC2460 Internet Protocol, Version 6 (IPv6) Specification [21]

Section in Document	Limitation
4. “IPv6 Extension Headers”	IPsec / “Authentication” and “Encapsulating Security Payload” (ESP) Headers are not supported.

Table 2-16 Deviations from IETF RFC2460

2.1.3.5.2 RFC3315 Dynamic Host Configuration Protocol for IPv6 (DHCPv6) [25]

Section in Document	Limitation
22.4. “Identity Association for Non-temporary Addresses Option”	At most one DHCPv6 address can be configured on each controller.

Table 2-17 Deviations from IETF RFC3315

2.1.3.5.3 RFC3484 Default Address Selection for Internet Protocol version 6 (IPv6) [46]

Section in Document	Limitation
5. “Source Address Selection”	Rule 4: Prefer home addresses. (no Mobile IP support) Rule 6: Prefer matching label.

Table 2-18 Deviations from IETF RFC3484

2.1.3.5.4 RFC4291 IP Version 6 Addressing Architecture [28]

Section in Document	Limitation
2.5.1. “Interface Identifiers”	Only 64 bit interface identifiers are supported: “For all unicast addresses, except those that start with the binary value 000, Interface IDs are required to be 64 bits long and to be constructed in Modified EUI-64 format.”

Table 2-19 Deviations from IETF RFC4291

2.1.3.5.5 RFC4443 Internet Control Message Protocol (ICMPv6) for the IPv6 Specification [30]

Section in Document	Limitation
3.1. “Destination Unreachable Message”	The IpV6 will not send Destination Unreachable Messages. (ICMP Message Type 1)

Table 2-20 Deviations from IETF RFC4443

2.1.3.5.6 RFC4704 The DHCPv6 Client FQDN option [32]

Section in Document	Limitation
General	The ‚Flags‘ field is filled with fixed values. No DNS update handling is implemented in any way.

Table 2-21 Deviations from IETF RFC4443

2.1.3.5.7 RFC4941 Privacy Extensions for Stateless Address Autoconfiguration in IPv6 [36]

RFC4941 “Privacy Extensions for Stateless Address Autoconfiguration” is currently not supported in MICROSAR4 Releases.

2.1.3.5.8 RFC3810 Multicast Listener Discovery Version 2 (MLDv2) for IPv6 [26]

RFC3810 “Multicast Listener Discovery Version 2 (MLDv2) for IPv6” is currently not supported in MICROSAR4 Releases.

2.1.4 Known Issues (low priority)

This section lists known issues that are not reported by the Vector issue reporting because they have low priority.

Compiler Warning “conditional expression is constant”

Some compilers may report warnings about conditional expressions that always evaluate to TRUE. These warnings are configuration dependent. If a TCPIP feature is enabled but not used by all IP instances/controllers a runtime check is required in order to decide whether the feature is enabled on a specific controller. If the feature is enabled on all controllers the condition is replaced by the constant value TRUE, so the compiler can eliminate the runtime check.

If a feature is disabled on all controllers, the code will be completely excluded from compilation.

2.1.5 Known Issues (IETF RFC Conformance)

The following items describe behavior of the Vector TCP/IP stack that slightly deviates from the behavior defined by the considered IETF RFCs.

These deviations have been identified using IETF RFC conformance tests.

Although this behavior may reduce TCP/IP performance in a few cases, it should not cause operational issues in currently known use cases.

IPv6 sends ICMP error messages in response to ICMP error messages

According to RFC4443 the TCPIP must not send an ICMPv6 error message as a result of receiving an ICMPv6 error message. But the IpV6 will do this if the received ICMPv6 error message is corrupt before the ICMPv6 header of the received error message.

ESCAN00071983

IPv6 stops reassembly of packet if the same fragment is received twice

According to IETF RFC 5722 IPv6 shall stop reassembly of a packet if two fragments overlap. But the RFC requirements are unclear about what shall be done if the same fragment is received twice. The IPv6 will also stop reassembly of a packet if a fragment is received twice.

ESCAN00084714

TCP backs off retransmission timer on fast retransmit

When a “Fast Retransmit” according to IETF RFC5681 has been sent, TCP will back-off the timer for the next regular retransmission.

ESCAN00084732

TCP uses too big cwnd if SYN or SYNACK is lost during active open

If a SYN/ACK segment gets lost during connection establishment, TCP assumes a bigger congestion window (CWND) than specified in IETF RFC5681. TCP will still respect the receive window of the remote host.

ESCAN00084734

TCP retransmits more than one segment after retransmit timeout expired

If the retransmit timer expires TCP may send more than one unacknowledged segment. The congestion window (CWND) is reduced as specified by IETF RFC6298.

ESCAN00084735

TCP does not echo timestamp option value of received SYN

If the TCP timestamp option according to IETF RFC1323 is enabled, the TCP will not send a timestamp echo value in a SYN segment. TCP sends timestamp echo values in other segments as specified by the RFC.

ESCAN00084729

TCP does not reply timestamp of earliest unacknowledged segment

If multiple TCP segments with different timestamp values are ACKed by the TCP the most recent timestamp value instead of the oldest value will be replied.

ESCAN00084731

Limitations of TCP User Timeout Option (RFC5482)

The user timeout value for a TCP socket will only be used if it is set after connection establishment.

Retransmitted segments do not contain a user timeout option.

Received user timeout values will only be considered while the connection is in an established state.

ESCAN00084733

TCP zero window probe intervals

Zero window probes are sent with an interval of the configured default retransmission timer until a window update is received.

ESCAN00084741

TCP sends window update with a value less than one MSS

TCP does not wait until the window can hold at least one MSS before sending a window update. This may affect TCP performance if very small chunks of data are received.

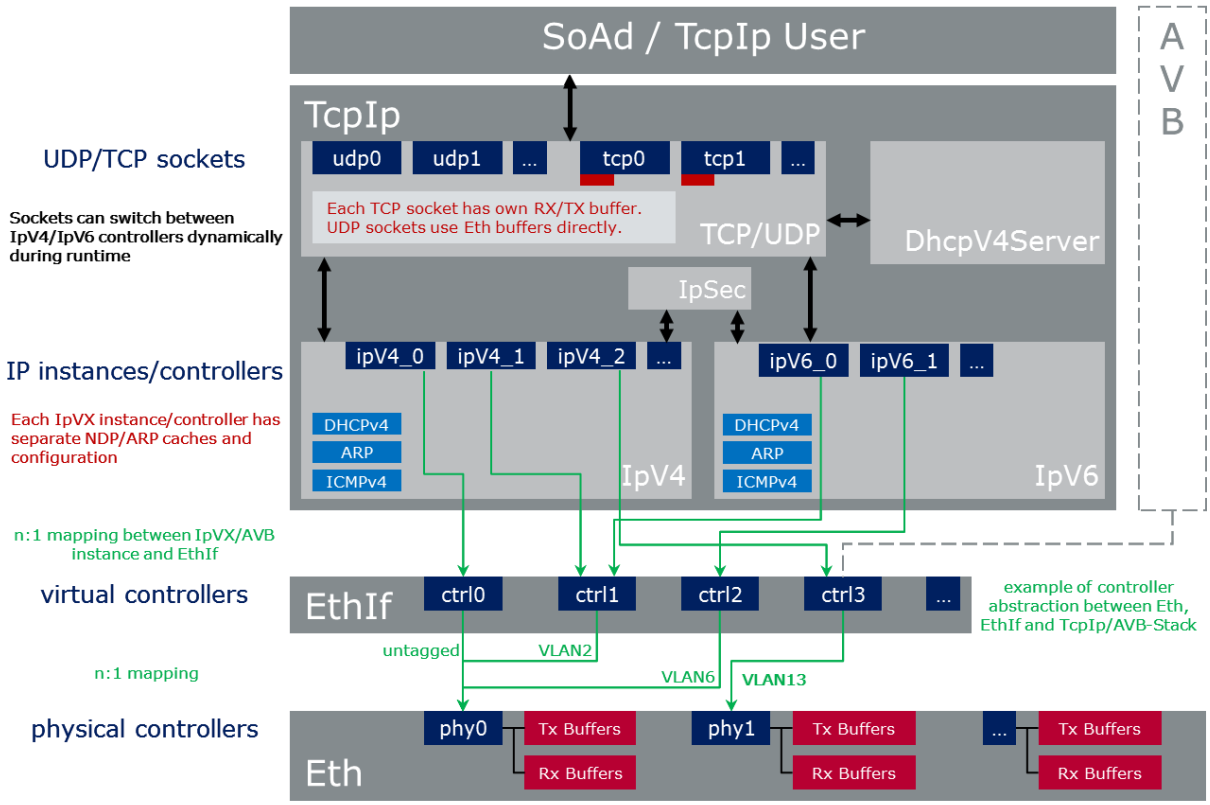
ESCAN00084744

TCP does not support simultaneous open of connection

The TCP specification allows that two TCP peers actively open a TCP connection simultaneously. This case is not used in practice and is therefore not supported.

ESCAN00084723

2.2 Structure of the TcpIp Stack



The TCPIP module provides UDP and TCP sockets to the upper layer (user).

A socket can be opened by the user during runtime via the API TcpIp_<Up>GetSocket by specifying the socket protocol (UDP/TCP) and the Internet Protocol version (IPv4/INET or IPv6/INET6).

After a socket has been opened it must be bound to a local IP address and port via the API TcpIp_Bind(). Local IP addresses are represented by identifiers (LocalAddrIds). A LocalAddrId represents an address of a specific IP instance/controller and is not shared between IP instances. IP instances operate independent from each other and use their own data structures like ARP/NDP caches and addresses.

Each IP instance/controller operates on exactly one virtual controller provided by the EthIf module.

Expert Knowledge
A virtual controller of the EthIf represents a physical Eth controller or a VLAN on a physical Eth controller.

2.2.1 IpV6

Each IPv6 instance has at least an auto configured link-local unicast address (see [34] 5.3. Creation of Link-Local Addresses) and an all-nodes multicast address. Additional unicast and multicast addresses may be configured.

2.2.2 IpV4

Each IPv4 instance has exactly one unicast and one broadcast address. Additionally one or more multicast addresses may be configured.

2.2.3 TLS

The TLS module has been adapted to work as a TcpIp plug-in. After requesting a TCP socket, the socket owner can call additional functions to request the usage of TLS on this socket.

- TcpIp_ChangeParameter(sock, TCPIP_PARAMID_V_USE_TLS, TRUE)
- TcpIp_ChangeParameter(sock, TCPIP_PARAMID_V_TLS_TXBUFSIZE, size)
- TcpIp_ChangeParameter(sock, TCPIP_PARAMID_V_TLS_RXBUFSIZE, size)
- TcpIp_Tls_SetClientCertInfo(sock, …) if client authentication shall be used
- TcpIp_ChangeParameter(sock, TCPIP_PARAMID_V_TLS_HEARTBEAT_MODE, TLS_HB_PEER_NOT_ALLOWED_TO_SEND)
- TcpIp_ChangeParameter(sock, TCPIP_PARAMID_V_TLS_HEARTBEAT_PERIOD_S, time)

The socket owner uses the normal TcpIp API to transmit and receive data.

Tls implements an additional callback to the TcpIp socket owner to exchange information on the validation of the server certificate chain. The callback ‘Tls Validation Result’ is an optional callback that is only used for sockets using TLS, and even for those TLS sockets the callback is optional.

For further information on TLS refer to [51].

2.2.4 DHCPv4 Server

In addition to a DHCPv4 client, the TcpIp optionally supports a DHCPv4 server according to IETF RFC2131 [20].

Independent DHCPv4 server instances can be configured for each IpV4 controller/VLAN.

According to AUTOSAR a DHCPv4 server instance optionally supports the assignment of IPv4 addresses based on the Ethernet Switch Port information. In this case the address assignment configuration is selected depending on the EthIf Switch Port on which the client DISCOVER/REQUEST message was received.

If no matching Ethernet Switch Port dependent address assignment configuration is found or configured a default address assignment configuration can be used.



Configuration in DaVinci Configurator Pro
The **TcpIpDhcpServerConfig** must be referenced by the **TcpIpDhcpServerConfigRef** parameter of the corresponding **TcpIp/TcpIpConfig/TcpIpCtrl**.

2.3 Initialization

The TcpIp is initialized by calling the `TcpIp_InitMemory()` service and afterwards calling the `TcpIp_Init()` service with the configuration as parameter.

The module supports the configuration variant ‘pre-compile’ in combination with the configuration of multiple variants (post-build variant support). Therefore, the address of the global configuration data structure of the module has to be passed to the `TcpIp_Init()` function.



Example

```
TcpIp_Init(TcpIp_Config_Ptr);
```

2.4 States

The TcpIp is operational after the initialization. Sockets can be requested immediately, but binding a socket to a local address is possible only after a local address could be assigned to the node previously.

IP addresses can be assigned / get assigned after a physical link to the network was detected.

2.5 Main Functions

The TcpIp can be configured to use either a single or three MainFunctions:

Single MainFunction (default)	Multiple MainFunctions
TcpIp_MainFunction	TcpIp_MainFunctionRx TcpIp_MainFunctionState TcpIp_MainFunctionTx

Table 2-22 MainFunctions of the TcpIp module

By default, only a single MainFunction needs to be called. This MainFunction processes state changes, transmission and reception of data for all submodules (IPv4, IPv6, UDP, TCP and DHCPv4 Server). The TcpIp_MainFunction wraps the calls to the Rx, Tx and State MainFunctions.

In order to optimize the response behavior for certain use cases it may be reasonable to split-up the Rx, Tx and state processing parts of the MainFunction. These separate MainFunctions may then be arranged individually inside the task.



There are some restrictions that apply to the mapping of the MainFunctions:

- All three MainFunctions must be called cyclically with the same period. Otherwise the timing of the protocols will be inaccurate.
- If the MainFunctions are mapped to different tasks these tasks must not interrupt each other.



Configuration in DaVinci Configurator Pro
See configuration parameters **TcpIpMainFunctionPeriod** and **TcpIpMainFunctionSplitEnabled**.

Depending on the value of TcpIpMainFunctionSplitEnabled one or three **Schedulable Entities** need to be mapped in the **Task Mapping** view of the Rte.

2.6 Timing Considerations

The processing of all TcpIp internal state changes is decoupled by at least a MainFunction cycle which can lead to delays in forwarding the events. This concerns both the forwarding of information to the upper layer and the processing of received TCP events such as RST or FIN, because the events are received in the TcpIp Rx-context but are processed in the next TcpIp State-context.

This means, after closing a Socket or receiving an RST, a MainFunction cycle must be waited until it is available for a new communication again.

2.7 Error Handling

2.7.1 Development Error Reporting

By default, development errors are reported to the DET using the service `Det_ReportError()` as specified in [2], if development error reporting is enabled (i.e. pre-compile parameter `TCPIP_DEV_ERROR_DETECT==STD_ON`).

If another module is used for development error reporting, the function prototype for reporting the error can be configured by the integrator but must have the same signature as the service `Det_ReportError()`.

The reported TCPIP ID is 170.

The submodules use the following instance IDs:

- TcpIp: 1
- IpV4: 2
- IpV6: 3
- DhcpV4Server: 4

The reported service IDs identify the services which are described in 4.2. The following table presents the service IDs and the related services:

Service ID	Service
0x01	TcpIp_Init
0x02	TcpIp_GetVersionInfo
0x03	TcpIp_GetSocketForUser (indirectly called by each configured socket owner)
0x04	TcpIp_Close
0x05	TcpIp_Bind
0x06	TcpIp_TcpConnect
0x07	TcpIp_TcpListen
0x08	TcpIp_TcpReceived
0x0A	TcpIp_RequestIpAddrAssignment
0x0B	TcpIp_ReleaseIpAddrAssignment
0x0D	TcpIp_DhcpWriteOption
0x0E	TcpIp_DhcpReadOption
0x0F	TcpIp_ChangeParameter
0x11	TcpIp_GetPhysAddr
0x12	TcpIp_UdpTransmit
0x13	TcpIp_TcpTransmit
0x14	TcpIp_RxIndication
0x15	TcpIp_MainFunction
0x16	TcpIp_GetRemotePhysAddr
0x17	TcpIp_GetCtrlIdx
0x19	TcpIp_DhcpV6ReadOption

Service ID	Service
0x1A	TcpIp_DhcpV6WriteOption
0x1C	TcpIp_GetNdpCacheEntries
0x1D	TcpIp_GetArpCacheEntries
0x45	TcpIp_GetAndResetMeasurementData
0x81	TcpIp_ProvideTxBufferInt
0x82	TcpIp_ProvideTxBuffer
0x83	TcpIp_TransmitTo
0x84	TcpIp_TransmitToInt
0x85	TcpIp_GetLocNetAddr
0x86	TcpIp_GetLocNetMask
0x87	TcpIp_SetDhcpHostNameOption
0x88	TcpIp_Tls_SetClientCertInfo
0x89	TcpIp_Tls_GetNvmBlockIdForUsedRootCert
0x8A	TcpIp_Tls_GetNvmBlockIdForUsedRootCert
0x8B	TcpIp_ClearARCache
0x8C	TcpIp_SendGratuitousArpReq
0x8D	TcpIp_IpSecSaEntryPairAdd
0x8E	TcpIp_IpSecSaEntryPairDelete
0x8F	TcpIp_IpSec_VInvokeSpdCallout
Services of the IpV4 submodule	
0x00	IpV4_Init
0x01	IpV4_Ip_MainFunction
0x03	IpV4_GetVersionInfo
0x04	IpV4_Ip_GetLocNetAddrCfgSrc
0x05	IpV4_Ip_SetNetAddr
0x07	IpV4_Ip_IsAddrIdAcceptable
0x20	IpV4_Ip_ProvideTxBuffer
0x21	IpV4_Ip_Transmit
0x22	IpV4_Ip_GetLocNetAddr
0x23	IpV4_Ip_GetLocNetMask
0x24	IpV4_Ip_AddrLossInd
0x25	IpV4_Ip_Dhcp_AddrInd
0x26	IpV4_Ip_ProvideNextTxBuffer
0x28	IpV4_Ip_RxIndication
0x29	IpV4_Ip_TxConfirmation

Service ID	Service
0x2A	IpV4_Ip_Cbk_TrcvLinkStateChg
0x30	IpV4_Ip_Init
0x31	IpV4_Ip_MainFunction
0x32	IpV4_Ip_Reset
0x33	IpV4_Ip_AddrConflictInd
0x34	IpV4_Ip_ResetSocket
0x35	IpV4_Ip_SetTimeToLive
0x36	IpV4_Ip_SetTypeOfService
0x37	IpV4_Ip_SetEthIfFramePrio
0x38	IpV4_Ip_RequestIpAddrAssignment
0x39	IpV4_GetLastDuplicateDhcpAddrDid
0x40	IpV4_Arp_RxIndication
0x50	IpV4_Arp_Init
0x51	IpV4_Arp_Reset
0x52	IpV4_Arp_MainFunction
0x53	IpV4_Arp_GetPhysicalAddress
0x54	IpV4_Arp_SendArpRequest
0x55	IpV4_Arp_MapIpToPhysMulticastAddr
0x56	IpV4_Arp_SendArpProbe
0x57	IpV4_Arp_SendArpAnnouncement
0x58	IpV4_Arp_SendGratuitousArpReq
0x60	IpV4_Icmp_SendEcho
0x70	IpV4_Icmp_Init
0x71	IpV4_Icmp_MainFunction
0x72	IpV4_Icmp_RxIndication
Services of the IpV6 submodule	
0x00	IpV6_GetVersionInfo
0x01	IpV6_InitMemory
0x02	IpV6_Init
0x03	IpV6_MainFunction
0x04	IpV6_ProvideTxBuffer
0x05	IpV6_Transmit

Service ID	Service
0x06	IpV6_RxIndication
0x07	IpV6_TxConfirmation
0x08	IpV6_Cbk_TrcvLinkStateChg
0x09	IpV6_SetAddress
0x0C	IpV6_SetTrafficClass
0x0D	IpV6_SetFlowLabel
0x0E	IpV6_IsValidDestinationAddress
0x0F	IpV6_GetLocalAddressCfgSrc
0x10	IpV6_GetCurrentHopLimit
0x11	IpV6_GetPhysAddr
0x12	IpV6_GetAsBcAddrId
0x13	IpV6_IsAddrIdAcceptable
0x14	IpV6_SetHopLimit
0x15	IpV6_SetEthIfFramePrio
0x16	IpV6_ResetSocket
0x17	IpV6_CancelTransmit
0x18	IpV6_GetLocalAddress
0x19	IpV6_GetDefaultRouterAddress
0x20	IpV6_Icmp_ProvideTxBuffer
0x21	IpV6_Icmp_TxEchoRequest
0x23	IpV6_Icmp_Transmit
0x24	IpV6_Icmp_MainFunction
0x30	IpV6_Ndp_MainFunction
0x40	IpV6_Mld_MainFunction
0x41	IpV6_Mld_Init

Table 2-23 Service IDs

The errors reported to DET are described in the following table:

Error Code	Description
0x01	TCPIP_E_NOTINIT
0x02	TCPIP_E_PARAM_POINTER
0x03	TCPIP_E_INV_ARG
0x04	TCPIP_E_NOBUFS
0x07	TCPIP_E_MSGSIZE
0x08	TCPIP_E_PROTOTYPE
0x09	TCPIP_E_ADDRINUSE
0x0A	TCPIP_E_ADDRNOTAVAIL
0x0B	TCPIP_E_ISCONN

Error Code	Description
0x0C	TCPIP_E_NOTCONN
0x0D	TCPIP_E_NOPROTOOPT
0x0E	TCPIP_E_AFNOSUPPORT
0x0F	TCPIP_E_INIT_FAILED
Services of the IpV4 submodule	
0x01	IPV4_E_NOT_INITIALIZED
0x02	IPV4_E_INV_POINTER
0x03	IPV4_E_INV_PARAM
0x04	IPV4_E_INV_CTRL_IDX
0x05	IPV4_E_INV_SOCK_IDX
0x06	IPV4_E_INV_CONFIG
0x07	IPV4_E_INV_ADDR_ID
Services of the IpV6 submodule	
0x01	IPV6_E_NOT_INITIALIZED
0x02	IPV6_E_INV_POINTER
0x03	IPV6_E_INV_SOCK_IDX
0x04	IPV6_E_INV_BUFF_IDX
0x05	IPV6_E_INV_CTRL_IDX
0x06	IPV6_E_INV_DATA_IDX
0x07	IPV6_E_INV_PARAM
0x08	IPV6_E_DLIST_BUFFER_EMPTY

Table 2-24 Errors reported to DET

2.7.2 Production Code Error Reporting

By default, production code related errors are reported to the DEM using the service `Dem_ReportErrorStatus()` as specified in [3], if production error reporting is enabled (i.e. at least one reference to a DEM event is configured in the TcpIp by referencing a DEM event using this EcuC path: `/ActiveEcuC/Dem/DemConfigSet/DemEventParameter`).

If another module is used for production code error reporting, the function prototype for reporting the error can be configured by the integrator, but must have the same signature as the service `Dem_ReportErrorStatus()`.

The errors reported to DEM are described in the following table:

Error Code	Description
TCPIP_E_DUPLICATE_DHCP_ADDR	An ECU's IPv4 address was used by another network node, i.e., the ECU received a frame which uses the same source IPv4 address but a different source MAC address. The DEM event is configured by creating a reference to a DEM event named `E DUPLICATE DHCP ADDR` in the EcuC-container: `/ActiveEcuC/TcpIp/TcpIpConfig/TcpIpCtrl_0_Fix/TcpIpCtrlDemEventParameterRefs` Refer to IpV4_GetLastDuplicateDhcpAddrDid for details on how to retrieve the related DID information.

Table 2-25 Errors reported to DEM



Note
From all references contained in the EcuC container `/ActiveEcuC/TcpIp/TcpIpConfig/TcpIpCtrl_0_Fix/TcpIpCtrlDemEventParameterRefs` the TcpIp currently only supports:
TCPIP_E_DUPLICATE_DHCP_ADDR

2.7.3 Other Error Reporting and Diagnostic Options

The TcpIp offers the possibility to track the following OEM-specific error conditions:

2.7.3.1 Tracking discarded ARP entries

TcpIp ARP (IPv4) offers the possibility to report an error in case an existing ARP entry was replaced by a new entry or a new entry could not be added to the ARP table because the maximum number of ARP entries has been reached.

Implementing this scenario is supported by the IPv4-subcomponent as follows:

- Turning on EcuC switch TcpIpArpDiscardedEntryHandling leads to the handling of the ARP-cache as ARP-table, which is never updated once it is full.
- Configuring the callback <Up_PhysAddrTableChg offers the possibility to track all new entries that are inserted into the ARP table.
- Configuring the callback <Up_PhysAddrTableEntryDiscarded> offers the possibility to track all IP addresses which are discarded because the ARP table is already full. (At the time when the address should have been resolved.)

2.7.3.2 Triggering the publishing of IP-address via diagnostic services

Some diagnostic services may demand that the ECU sends a gratuitous ARP request packet to publish its IP-address. This possibility is offered by the function TcpIp_SendGratuitousArpReq.

2.7.3.3 Get DHCP Status for IPv4

TcpIp DHCPv4 offers the possibility to retrieve the status of the DHCP handshaking for an IPv4 address that has DHCP configured as address assignment method.
This possibility is offered by the function TcpIp_DhcpV4_GetStatus.

2.7.3.4 IPv6 Duplicate Address Detection Conflict

TcpIp supports the implementation of a DEM event that is equivalent to the runtime error TCPIP_E_DADCONFLICT described in [1].

Setting the DEM event to ‘active’ can be done by configuring the callback DADAddressConflict() (Refer to section 4.5.1.8).

In order to ‘heal’ the DEM event, the LocalAddrId parameter of the latter callback has to be stored and the state-change of the affected address back to a valid IP address has to be tracked using the configurable callback LocalIpAddrAssignmentChg()-callback (Refer to EcuC item: /MICROSAR/TcpIp/TcpIpConfig/TcpIpSocketOwnerConfig/TcpIpSocketOwner/TcpIpSocketOwnerLocalIpAddrAssignmentChgName).

3 Integration

This chapter gives necessary information for the integration of the MICROSAR TCPIP into an application environment of an ECU.

3.1 Scope of Delivery

The delivery of the TCPIP contains the files which are described in the chapters 3.1.1 and 3.1.2:

3.1.1 Static Files

File Name	Description
TcpIp Static Files	
TcpIp.(c|h)	Implementation/Declaration of public TcpIp APIs
TcpIp_Cbk.h	Declaration of public call-back APIs
TcpIp_Types.h	Types header for ASR API of TcpIp
TcpIp_Tcp.(c|h)	Implementation of TCP (internal functions)
TcpIp_Tcp_Cbk.h	Implementation of TCP (call-back function declarations)
TcpIp_Udp.(c|h)	Implementation of UDP (internal functions)
TcpIp_Udp_Cbk.h	Implementation of UDP (call-back function declarations)
TcpIp_Priv.(c|h)	Implementation of internal/private TcpIp functions
TcpIp_Priv_Types.h	TcpIp internal Type definitions
TcpIp_IpSec.(c|h)	Implementation of IPsec
IpV4 Static Files	
TcpIp_IpV4.(c|h)	Implementation of IPv4 (internal functions)
TcpIp_IpV4_Cbk.h	Implementation of IPv4 (call-back function declarations)
TcpIp_IpV4_Types.h	Internal type definitions of the IpV4 submodule
TcpIp_Arp.(c|h)	Implementation of ARP for IPv4 (internal functions)
TcpIp_Arp_Cbk.h	Implementation of ARP for IPv4 (call-back function declarations)
TcpIp_DhcpV4.(c|h)	Implementation of DHCP Client for IPv4 (internal functions)
TcpIp_DhcpV4_Cbk.h	Implementation of DHCP Client for IPv4 (call-back function declarations)
TcpIp_IcmpV4.(c|h)	Implementation of ICMP for IPv4 (internal functions)
TcpIp_IcmpV4_Cbk.h	Implementation of ICMP for IPv4 (call-back function declarations)
TcpIp_IpV4_Priv.(c|h)	Implementation of internal/private functions of the IpV4 submodule
DhcpV4Server Static Files	
TcpIp_DhcpV4Server.(c|h)	Implementation of DHCP Server for IPv4
IpV6 Static Files	
TcpIp_IpV6.(c|h)	Implementation of IPv6 (internal functions)
TcpIp_IpV6_Cbk.h	Implementation of IPv6 (call-back function declarations)
TcpIp_IpV6_Types.h	Internal type definitions of the IpV6 submodule

File Name	Description
TcpIp_IcmpV6.(c|h)	Implementation of ICMP for IPv6 (internal functions)
TcpIp_Ndp.(c|h)	Implementation of NDP for IPv6 (internal functions)
TcpIp_Mld.(c|h)	Implementation of MLDv2 for IPv6 (internal functions)
TcpIp_DhcpV6.(c|h)	Implementation of DHCP Client for IPv6 (internal functions)
TcpIp_DhcpV6_Cbk.h	Implementation of DHCP Client for IPv6 (call-back function declarations)
TcpIp_IpV6_Priv.(c|h)	Implementation of internal/private functions of the IpV6 submodule

Table 3-1 Static files

3.1.2 Dynamic Files

The dynamic files are generated by the configuration tool DaVinci Configurator Pro.

File Name	Description
Dynamic Files (always generated if TcpIp module exists in configuration)	
TcpIp_Cfg.h	TcpIp Pre-compile time parameter configuration
TcpIp_Lcfg.(c|h)	TcpIp Link-time parameter configuration
TcpIp_PBcfg.(c|h)	TcpIp Post-Build parameter configuration (empty file) Post-build is currently not supported by the TcpIp.

Table 3-2 Generated files

3.2 Critical Sections

The TCPIP calls service functions of upper layers in order to prevent interruption of critical sections.

These service functions have to be provided (normally by the Schedule Manager) and configured accordingly.

To ensure data consistency and a correct function of the TCPIP module the exclusive areas `TCPIP_EXCLUSIVE_AREA_0`, `IPV4_EXCLUSIVE_AREA_0` and `IPV6_EXCLUSIVE_AREA_0` have to be provided during the integration.

Considering the timing behavior of your system (e.g. depending on the CPU load of your system, priorities and interruptibility of interrupts and OS tasks and their jitter and delay times) the integrator has to choose and configure a critical section solution in such way that it is ensured that the API functions do not interrupt each other.

It is recommended to use the functions `SuspendAllInterrupts()` and `ResumeAllInterrupts`() (global interrupt locks) for the used exclusive areas to ensure data consistency.

3.3 Preemption

TcpIp expects that its own main functions do not interrupt each other. Additionally, it expects that an own main function does not interrupt main functions of related modules and is not interrupted by main functions of related modules.



Caution
Consider main function expectations when using preemptive tasks.

4 API Description

4.1 Type Definitions

The types defined by the TCPIP are described in this chapter.

Type Name	C-Type	Description	Value Range
TcpIp_ConfigType		Configuration data structure of the TcpIp module.	Configuration data structure of the TcpIp module.
TcpIp_DomainType	uint16	TcpIp address families.	`TCPIP_AF_INET (0x02)` Use IPv4
			`TCPIP_AF_INET6 (0x1c)` Use IPv6
TcpIp_ProtocolType	Enum	Protocol type used by a socket.	`TCPIP_IPPROTO_TCP (0x06)` Use TCP
			`TCPIP_IPPROTO_UDP (0x11)` Use UDP
TcpIp_LocalAddrIdType	uint8	Address identification type for unique identification of a local IP address and EthIf Controller configured in the TcpIp module.	
TcpIp_SocketIdType	uint8, uint16	socket identifier type for unique identification of a TcpIp stack socket. TCPIP_SOCKETID_INVALID shall specify an invalid socket handle	
			`TCPIP_STATE_ONLINE`

Type Name	C-Type	Description	Value Range
TcpIp_StateType	Enum	Specifies the TcpIp state for a specific EthIf controller.	TCP/IP stack state for a specific EthIf controller is ONLINE, i.e. communication via at least one IP address is possible.
			TCPIP_STATE_ONHOLD TCP/IP stack state for a specific EthIf controller is ONHOLD, i.e. no communication is currently possible (e.g. link down).
			TCPIP_STATE_OFFLINE TCP/IP stack state for a specific EthIf controller is OFFLINE, i.e. no communication is possible.
			TCPIP_STATE_STARTUP TCP/IP stack state for a specific EthIf controller is STARTUP, i.e. IP address assignment in progress or ready for manual start, communication is currently not possible.
			TCPIP_STATE_SHUTDOWN TCP/IP stack state for a specific EthIf controller is SHUTDOWN, i.e. release of resources using the EthIf controller, release of IP address assignment.
TcpIp_IpAddrStateType	Enum	Specifies the state of local IP address assignment	TCPIP_IPADDR_STATE_ASSIGNED local IP address is assigned
			TCPIP_IPADDR_STATE_ONHOLD local IP address is assigned, but cannot be used as the network is not active
			TCPIP_IPADDR_STATE_UNASSIGNED local IP address is unassigned
TcpIp_EventType	Enum	Events reported by TcpIp.	TCPIP_TCP_RESET TCP connection was reset, TCP socket and all related resources have been released.
			TCPIP_TCP_CLOSED

Type Name	C-Type	Description	Value Range
			TCP connection was closed successfully, TCP socket and all related resources have been released.
			`TCPIP_TCP_FIN_RECEIVED` A FIN signal was received on the TCP connection, TCP socket is still valid.
			`TCPIP_UDP_CLOSED` UDP socket and all related resources have been released.
TcpIp_IpAddrAssignmentType	Enum	Specification of IPv4/IPv6 address assignment policy.	`TCPIP_IPADDR_ASSIGNMENT_STATIC` Static configured IPv4/IPv6 address.
			`TCPIP_IPADDR_ASSIGNMENT_LINKLOCAL_DOIP` Linklocal IPv4/IPv6 address assignment using DoIP parameters.
			`TCPIP_IPADDR_ASSIGNMENT_DHCP` Dynamic configured IPv4/IPv6 address by DHCP.
			`TCPIP_IPADDR_ASSIGNMENT_LINKLOCAL` Linklocal IPv4/IPv6 address assignment.
			`TCPIP_IPADDR_ASSIGNMENT_IPV6_ROUTER` Dynamic configured IPv4/IPv6 address by Router Advertisement.
TcpIp_ReturnType	Enum	TcpIp specific return type.	`TCPIP_OK` Operation completed successfully.
			`TCPIP_E_NOT_OK` Operation failed.
			`TCPIP_E_PHYS_ADDR_MISS` Operation failed because of an ARP/NDP cache miss.
TcpIp_ParamIdType	uint8	Type for the specification of all supported Parameter IDs.	`TCPIP_PARAMID_TCP_RXWND_MAX  0x00` Specifies the maximum TCP receive window for the socket.
			`TCPIP_PARAMID_FRAMEPRIO  (0x01)`

Type Name	C-Type	Description	Value Range
			Specifies the frame priority for outgoing frames on the socket.
			TCPIP_PARAMID_TCP_NAGLE (0x02) Specifies if the Nagle Algorithm according to IETF RFC 896 is enabled or not.
			TCPIP_PARAMID_TCP_KEEPALIVE (0x03) Specifies if TCP Keep Alive Probes are sent on the socket connection.
			TCPIP_PARAMID_TTL (0x04) Specifies the time to live value for outgoing frames on the socket. For IPv6 this parameter specifies the value of the HopLimit field used in the IPv6 header.
			TCPIP_PARAMID_TCP_KEEPALIVE_TIME (0x05) Specifies the time in [s] between the last data packet sent (simple ACKs are not considered data) and the first keepalive probe.
			TCPIP_PARAMID_TCP_KEEPALIVE_PROBES_MAX (0x06) Specifies the maximum number of times that a keepalive probe is retransmitted.
			TCPIP_PARAMID_TCP_KEEPALIVE_INTERVAL (0x07) Specifies the interval in [s] between subsequent keepalive probes.
			TCPIP_PARAMID_VENDOR_SPECIFIC (0x80) Start of vendor specific range of parameter IDs.
TcpIpIpAddrWildcardType	uint32	IP address wildcard.	TCPIP_IPADDR_ANY defines the value used as wildcard
TcpIpIp6AddrWildcardType	uint32	IP6 address wildcard.	TCPIP_IP6ADDR_ANY defines the value used as wildcard for all IP6 address parts
TcpIpPortWildcardType	uint16	Port wildcard.	TCPIP_PORT_ANY defines the value used as wildcard

Type Name	C-Type	Description	Value Range
TcpIpLocalAddrIdWildcardType	uint8	LocalAddrId wildcard.	`TCPIP_LOCALADDRID_ANY` defines the value used as wildcard
TcpIp_ArpCacheEntryType TcpIp_NdpCacheEntryType	struct	Represents an entry inside the ARP/NDP cache	`TCPIP_(NDP|ARP)_ENTRY_STATIC (0x00)` defines the value used for static NDP/ARP cache entries used in TcpIp_Get(Ndp|Arp)CacheEntries() API
			`TCPIP_(NDP|ARP)_ENTRY_VALID (0x01)` defines the value used for valid NDP/ARP cache entries used in TcpIp_Get(Ndp|Arp)CacheEntries() API
			`TCPIP_(NDP|ARP)_ENTRY_STALE (0x02)` defines the value used for stale NDP/ARP cache entries used in TcpIp_Get(Ndp|Arp)CacheEntries() API
TcpIp_DhcpEventType	uint8	Type of the current DHCPv4/DHCPv6 event.	`TCPIP_DHCP_EVENT_TX_DISCOVER_SOLICIT (0x00)` Discover (DHCPv4) or Solicit (DHCPv6) message will be transmitted.
			`TCPIP_DHCP_EVENT_RX_OFFER_ADVERTISE (0x01)` Offer (DHCPv4) or Advertise (DHCPv6) message was received.
			`TCPIP_DHCP_EVENT_TX_REQUEST (0x02)` Request message (DHCPv4, DHCPv6) message will be transmitted.
			`TCPIP_DHCP_EVENT_RX_ACK_REPLY (0x03)` Acknowledge (DHCPv4) or Reply (DHCPv6) message was received.
			`TCPIP_DHCP_EVENT_TX_V6_CONFIRM (0x04)` Confirm (DHCPv6) message will be transmitted.
			`TCPIP_DHCP_EVENT_TX_V6_RENEW (0x05)` Renew (DHCPv6) message will be transmitted.
			`TCPIP_DHCP_EVENT_TX_V6_REBIND (0x06)`

© 2021 Vector Informatik GmbH

Type Name	C-Type	Description	Value Range
			Rebind (DHCPv6) message will be transmitted.
			`TCPIP_DHCP_EVENT_TX_V6_RELEASE (0x07)` Release (DHCPv6) message will be transmitted.
			`TCPIP_DHCP_EVENT_TX_V6_DECLINE (0x08)` Acknowledge Decline (DHCPv6) message will be transmitted.
TcpIp_MeasurementIdxType	uint8	Represents the index to select specific measurement data	`TCPIP_MEAS_DROP_TCP (0x01)` Measurement index of dropped TCP packets caused by invalid destination IP address or TCP port.
			`TCPIP_MEAS_DROP_UDP (0x02)` Measurement index of dropped UDP packets caused by invalid destination IP address or UDP port.
			`TCPIP_MEAS_DROP_IPV4 (0x03)` Measurement index of dropped datagrams caused by invalid destination IPV4 address.
			`TCPIP_MEAS_DROP_IPV6 (0x04)` Measurement index of dropped datagrams caused by invalid destination IPV6 address.
			`TCPIP_MEAS_VENDOR_SPECIFIC_DROP_LAYER3_IPV6 (0x80)` Measurement index of dropped datagrams caused by invalid IPv6 packet.
			`TCPIP_MEAS_VENDOR_SPECIFIC_DROP_LAYER4 (0x81)` Measurement index of dropped datagram caused by invalid TCP/UDP packets.
			`TCPIP_MEAS_VENDOR_SPECIFIC_NR_SA_PAIRS (0x82)` Measurement index of number of valid SA entry pairs present in the SAD. (Only applicable if IPsec is enabled)

Type Name	C-Type	Description	Value Range
			TCPIP_MEAS_VENDOR_SPECIFIC_SA_PAIRS (0x83) Measurement index of statuses of valid SA entry pairs in the SAD. (Only applicable if IPsec is enabled)
			TCPIP_MEAS_VENDOR_SPECIFIC_DROP_INSUFF_TCP_TX_BUFFER (0x84) Measurement index of dropped datagram caused by insufficient TCP Tx buffer.
			TCPIP_MEAS_VENDOR_SPECIFIC_DROP_INSUFF_TCP_RX_BUFFER (0x85) Measurement index of dropped datagram caused by insufficient TCP Rx buffer.
			TCPIP_MEAS_VENDOR_SPECIFIC_DROP_INSUFF_IPV4_FRAGMENT_RX_BUFFER (0x86) Measurement index of dropped datagram caused by insufficient IpV4 reassembly buffer.
			TCPIP_MEAS_ALL (0xFF) Represents all measurement indexes, only in case of reset.
TcpIp_IpSecPolicyType	uint8	IPsec Security Policy Type	TCPIP_IPSEC_POLICY_BYPASS (0x00) Defines the bypass security policy which means the IP packet will not be protected by IPsec.
			TCPIP_IPSEC_POLICY_PROTECT (0x01) Defines the protect security policy which means the IP packet will be protected by IPsec.
			TCPIP_IPSEC_POLICY_OPTIONAL (0x02) Defines the optional security policy which means the IP packet will be protected by IPsec if possible (if SA entry is present) or be unprotected otherwise.
			TCPIP_IPSEC_POLICY_DISCARD (0x03)

Type Name	C-Type	Description	Value Range
			Defines the discard security policy which means the IP packet will be discarded without further processing.
TcpIp_IpSecHeaderType	uint8	IPsec Headers	`TCPIP_IPSEC_HDR_AH_ESP (0x01)` Both Authentication header and ESP header shall be used for IPsec protected traffic. (NOT supported)
			`TCPIP_IPSEC_HDR_AH (0x02)` Only Authentication header shall be used for IPsec protected traffic.
			`TCPIP_IPSEC_HDR_ESP (0x03)` Only ESP header shall be used for IPsec protected traffic.
TcpIp_IpSecEventType	uint8	IPsec auditable events	`TCPIP_IPSEC_NO_EVENT (0x00)` No IPsec Auditable event.
			`TCPIP_IPSEC_EVENT_INVALID_HDR (0x01)` Invalid IPsec header.
			`TCPIP_IPSEC_EVENT_DISCARD_POLICY (0x02)` Discard IPsec security policy.
			`TCPIP_IPSEC_EVENT_SEQ_OVERFLOW (0x03)` AH sequence number overflow.
			`TCPIP_IPSEC_EVENT_ANTIREPLAY (0x04)` AH anti replay attack.
			`TCPIP_IPSEC_EVENT_VERIFY_FAILED (0x05)` AH authentication data verification failed.
			`TCPIP_IPSEC_EVENT_GEN_FAILED (0x06)` AH authentication data generation failed.
			`TCPIP_IPSEC_EVENT_FRAGMENT (0x07)` Fragmented IPsec IP packet.
			`TCPIP_IPSEC_EVENT_SAENTRY_NOTFOUND (0x08)`

© 2021 Vector Informatik GmbH

Type Name	C-Type	Description	Value Range
			SA entry lookup failure.
TcpIp_IpSecIntegrityTransformType	uint16	Integrity algorithm transform Ids	Refer Section Transform Type 3 - Integrity Algorithm Transform IDs in [50]
TcpIp_IpSecEncrTransformType	uint16	Encryption algorithm transform Ids	Refer Section Transform Type 1 - Encryption Algorithm Transform IDs in [50]

Table 4-1 Type definitions

TcpIp_SockAddrType

Generic structure used by APIs to specify an IP address. (A specific address type can be derived from this structure via a cast to the specific struct type.)

Struct Element Name	C-Type	Description
domain	TcpIp_DomainType (uint16)	This is the code for the address format of this address

Table 4-2 TcpIp_SockAddrType

TcpIp_SockAddrInetType

This structure defines an IPv4 address type which can be derived from the generic address structure via cast.

Struct Element Name	C-Type	Description
domain	TcpIp_DomainType (uint16)	This is the code for the address format of this address
port	uint16	port number
addr	uint32[1]	IPv4 address in network byte order

Table 4-3 TcpIp_SockAddrInetType

TcpIp_SockAddrInet6Type

This structure defines an IPv6 address type which can be derived from the generic address structure via cast.

Struct Element Name	C-Type	Description
domain	TcpIp_DomainType (uint16)	This is the code for the address format of this address
port	uint16	port number
addr	uint32[4]	IPv6 address in network byte order

Table 4-4 TcpIp_SockAddrInet6Type

TcpIp_IpSecKeyMaterialInfoType

This structure defines the parameters which describes the keying material for an SA entry Pair.

Struct Element Name	C-Type	Description
InboundKeyMatPtr	uint8*	Key material for inbound job
OutboundKeyMatPtr	uint8*	Key material for outbound job
InboundKeyMatLen	uint16	Key length for inbound job
OutboundKeyMatLen	uint16	Key length for outbound job

Table 4-5 TcpIp_IpSecKeyMaterialInfoType

TcpIp_IpSecTransformInfoType

This structure defines the parameters which describes the SA entry pair.

Struct Element Name	C-Type	Description
IntegTransformIdent	TcpIp_IpSecIntegrityTransformType	Identifier for the integrity transform
EncryptTransformIdent	TcpIp_IpSecEncrTransformType	identifier for the encryption transform
EsnEnabled	boolean	Esn support flag

Table 4-6 TcpIp_IpSecTransformInfoType

TcpIp_IpSecTrafficSelectorType

This structure defines the traffic selectors for the SA entry pair.

Struct Element Name	C-Type	Description
RemoteAddr	TcpIp_SockAddrType	Remote Ip address (in network byte order)
LocalPortRangeStart	uint16	Local port range start (in host byte order)
LocalPortRangeEnd	uint16	Local port range start (in host byte order)
RemotePortRangeStart	uint16	Remote port range start (in host byte order)
RemotePortRangeEnd	uint16	Remote port range start (in host byte order)

Struct Element Name	C-Type	Description
IpProtocol	TcpIp_IpProtocolType	Upper layer protocols supported

Table 4-7 TcpIp_IpSecTrafficSelectorType

TcpIp_IpSecSaInfoType

This structure defines the parameters of the Security Association (SA) entry pair.

Struct Element Name	C-Type	Description
KeyMaterial	TcpIp_IpSecKeyMaterialInfoType	Key material for the SA Entry pair
SecurityTransform	boolean	Specifies the security transforms configured of the SA entry pair
InboundSpi	uint32	Spi of the inbound traffic
OutboundSpi	uint32	Spi of the outbound traffic
TrafficSelectors	TcpIp_IpSecTrafficSelectorType*	List of traffic selectors specified
NumTrafficSelectors	uint16	Number of traffic selectors specified
IpsecHdrType	TcpIp_IpSecHeaderType	IPsec header type

Table 4-8 TcpIp_IpSecSaInfoType

TcpIp_IpSecSaStatusType

Type Name	C-Type	Description
InboundSpi	uint32	Spi of SA for inbound traffic
OutboundSpi	uint32	Spi of SA for outbound traffic
IncomingPktCnt	uint32	Count of incoming IPsec packets
OutgoingPktCnt	uint32	Count of outgoing IPsec packets
SecsSinceLastRx	uint16	Seconds elapsed since last IPsec packet Rx [0: 0xFFFF] (No wraparound)
SecsSinceLastTx	uint16	Seconds elapsed since last IPsec packet Tx [0: 0xFFFF] (No wraparound)
IpsecProtocol	TcpIp_IpSecHeaderType	IPsec header type

Table 4-9 TcpIp_IpSecSaStatusType

4.2 Services provided by TCPIP

4.2.1 TcpIp_GetVersionInfo

Prototype	
`void TcpIp_GetVersionInfo (Std_VersionInfoType *versioninfo)`	
Parameter	
versioninfo [in]	pointer for version information

Return code	
void	void
Functional Description	
Get Tcp/Ip Version.	
Particularities and Limitations	
The module version is given decimal coded.	
Call context	
> task level	

Table 4-10 TcpIp_GetVersionInfo

4.2.2 TcpIp_InitMemory

Prototype	
`void TcpIp_InitMemory (void)`	
Parameter	
void [in]	none
Return code	
void	void
Functional Description	
Initialize the TcpIp-internal global module state variable.	
Particularities and Limitations	
Call context	
> initialization	

Table 4-11 TcpIp_InitMemory

4.2.3 TcpIp_Init

Prototype	
`void TcpIp_Init (const TcpIp_ConfigType *ConfigPtr)`	
Parameter	
ConfigPtr [in]	pointer to module configuration
Return code	
void	void
Functional Description	
Initialize the TcpIp module. Calls all TcpIp internal init functions and sets state to 'initialized'.	
Particularities and Limitations	
Call context	
> initialization	

Table 4-12 TcpIp_Init

4.2.4 TcpIp_<Up>GetSocket

Prototype	
`Std_ReturnType TcpIp_<Up>GetSocket(TcpIp_DomainType Domain, TcpIp_ProtocolType Protocol, TcpIp_SocketIdType *SocketIdPtr)`	
Parameter	
Domain [in]	Domain / address family that will be used with the socket. TCPIP_AF_INET (IPv4) or TCPIP_AF_INET6 (IPv6).
Protocol [in]	Transport layer protocol that will be used with the socket. TCPIP_IPPROTO_TCP or TCPIP_IPPROTO_UDP
SocketIdPtr [out]	Pointer to socket identifier representing the requested socket. This socket identifier must be provided for all further API calls which require a SocketId.
Return code	
Std_ReturnType	> E_OK The request has been accepted > E_NOT_OK The request has not been accepted: no free socket
Functional Description	
Request a socket (TCP or UDP). By this API service the TCP/IP stack is requested to allocate a new socket.	
Particularities and Limitations	
SocketIdPtr is only valid if return value is E_OK. Each accepted incoming TCP connection also allocates a socket resource. Server sockets are special because they do not need any rx and tx buffer and a lot less socket description parameters.	
Call context	
> task level	

Table 4-13 TcpIp_<Up>GetSocket

4.2.5 TcpIp_GetSocketForUser

Prototype	
`Std_ReturnType` **`TcpIp_GetSocketForUser`**`(TcpIp_DomainType Domain, TcpIp_ProtocolType Protocol, TcpIp_SocketIdType *SocketIdPtr, TcpIp_SocketOwnerConfigIterType SocketOwnerIdx)`	
Parameter	
Domain [in]	Domain / address family that will be used with the socket. TCPIP_AF_INET (IPv4) or TCPIP_AF_INET6 (IPv6).
Protocol [in]	Transport layer protocol that will be used with the socket. TCPIP_IPPROTO_TCP or TCPIP_IPPROTO_UDP
SocketIdPtr [out]	Pointer to socket identifier representing the requested socket. This socket identifier must be provided for all further API calls which require a SocketId.
SocketOwnerIdx [in]	Index of a socket owner.
Return code	
Std_ReturnType	> E_OK The request has been accepted > E_NOT_OK The request has not been accepted: no free socket
Functional Description	
Request a socket (TCP or UDP). By this API service the TCP/IP stack is requested to allocate a new socket.	
Particularities and Limitations	
SocketIdPtr is only valid if return value is E_OK. Each accepted incoming TCP connection also allocates a socket resource. Server sockets are special because they do not need any rx and tx buffer and a lot less socket description parameters.	
Call context	
> task level	

Table 4-14 TcpIp_GetSocketForUser

4.2.6 TcpIp_ChangeParameter

Prototype	
`Std_ReturnType` **`TcpIp_ChangeParameter`** `(TcpIp_SocketIdType SocketId, TcpIp_ParamIdType ParameterId, uint8 *ParameterValue)`	
Parameter	
SockHnd [in]	socket handle
ParameterId [in]	parameter identification
ParameterValue [in]	parameter value
Return code	
Std_ReturnType	> E_OK parameter changed > E_NOT_OK parameter change failed
Functional Description	
change socket parameter configuration	
Particularities and Limitations	
Call context	
> task level	

Table 4-15 TcpIp_ChangeParameter

4.2.7 TcpIp_Bind

Prototype	
`Std_ReturnType **TcpIp_Bind** (TcpIp_SocketIdType SocketId, TcpIp_LocalAddrIdType LocalAddrId, uint16 *PortPtr)`	
Parameter	
SocketId [in]	Socket identifier of the related local socket resource.
LocalAddrId [in]	IP address identifier representing the local IP address and EthIf controller to bind the socket to.
PortPtr [in, out]	Pointer to memory where the local port to which the socket shall be bound is specified. In case the parameter is specified as TCPIP_PORT_ANY, the TCP/IP stack shall choose the local port automatically from the range 49152 to 65535 and shall update the parameter to the chosen value.
Return code	
Std_ReturnType	> E_OK The request has been accepted > E_NOT_OK The request has not been accepted (e.g. address in use)
Functional Description	
By this API service the TCP/IP stack is requested to bind a UDP or TCP socket to a local resource.	
Particularities and Limitations	
Please ensure that the input parameter SocketId of the TcpIp_Bind function is valid and must be requested before the TcpIp_Bind call via the API TcpIp_<Up>GetSocket or TcpIp_GetSockerForUser.	
Call context	
> task level	

Table 4-16 TcpIp_Bind

4.2.8 TcpIp_UdpTransmit

Prototype	
`TcpIp_ReturnType` **`TcpIp_UdpTransmit`** `(TcpIp_SocketIdType SocketId, uint8 *DataPtr, TcpIp_SockAddrType *RemoteAddrPtr, uint16 TotalLength)`	
Parameter	
SocketId [in]	socket index
DataPtr [in]	Pointer to a linear buffer of TotalLength bytes containing the data to be transmitted. In case DataPtr is a NULL_PTR, TcpIp shall retrieve data from SoAd via callback SoAd_CopyTxData().
RemoteAddrPtr [in]	IP address and port of the remote host to transmit to.
TotalLength [in]	indicates the payload size of the UDP datagram.
Return code	
TcpIp_ReturnType	> TCPIP_OK UDP message has been forwarded to EthIf for transmission. > TCPIP_E_NOT_OK UDP message could not be sent because of a permanent error, e.g. message is too long. > TCPIP_E_PHYS_ADDR_MISS UDP message could not be sent because of an ARP cache miss, ARP request has been sent and upper layer may retry transmission by calling this function later again.
Functional Description	
Transmit UDP message. This service transmits data via UDP to a remote node. The transmission of the data is immediately performed with this function call by forwarding it to EthIf.	
Particularities and Limitations	
This function will return TCPIP_E_PHYS_ADDR_MISS in case the physical address could not be resolved or no Ethernet TxBuffer was available. Differing between 'phy addr resolution is missing' and 'no buffer available' does not imply any useful information for the user. This function may not be called reentrant from one socket owner.	
Call context	
> task level	

Table 4-17 TcpIp_UdpTransmit

4.2.9 TcpIp_TcpListen

Prototype	
`Std_ReturnType` **`TcpIp_TcpListen`** `(TcpIp_SocketIdType SocketId, uint16 MaxChannels)`	
Parameter	
SocketId [in]	Socket identifier of the related local socket resource.
MaxChannels [in]	Maximum number of new parallel connections established on this listen connection.
Return code	
Std_ReturnType	> E_OK The request has been accepted > E_NOT_OK The request has not been accepted
Functional Description	
Set TCP socket into listen mode. By this API service the TCP/IP stack is requested to listen on the TCP socket specified by the socket identifier.	
Particularities and Limitations	
Call context	
> task level	

Table 4-18 TcpIp_TcpListen

4.2.10 TcpIp_TcpConnect

Prototype	
`Std_ReturnType` **`TcpIp_TcpConnect`** `(TcpIp_SocketIdType SocketId, const TcpIp_SockAddrType *RemoteAddrPtr)`	
Parameter	
SocketId [in]	Socket identifier of the related local socket resource
RemoteAddrPtr [in]	IP address and port of the remote host to connect to
Return code	
Std_ReturnType	> E_OK Connect could be triggered > E_NOT_OK Connect could be triggered
Functional Description	
By this API service the TCP/IP stack is requested to establish a TCP connection to the configured peer.	
Particularities and Limitations	
Call context	
> task level	

Table 4-19 TcpIp_TcpConnect

4.2.11 TcpIp_TcpTransmit

Prototype	
`Std_ReturnType` **`TcpIp_TcpTransmit`** `(TcpIp_SocketIdType SocketId, uint8 *DataPtr, uint32 AvailableLength, boolean ForceRetrieve)`	
Parameter	
SocketId [in]	socket index
DataPtr [in]	Pointer to a linear buffer of AvailableLength bytes containing the data to be transmitted. In case DataPtr is a NULL_PTR, TcpIp shall retrieve data from SoAd via callback SoAd_CopyTxData().
AvailableLength [in]	Available data for transmission in bytes
ForceRetrieve [in]	This parameter is only valid if DataPtr is a NULL_PTR. Indicates how the TCP/IP stack retrieves data from SoAd if DataPtr is a NULL_PTR. TRUE: the whole data indicated by availableLength shall be retrieved from the upper layer via one or multiple SoAd_CopyTxData() calls within the context of this transmit function. FALSE: The TCP/IP stack may retrieve up to availableLength data from the upper layer. It is allowed to retrieve less than availableLength bytes. Note: Not retrieved data will be provided by SoAd with the next call to TcpIp_TcpTransmit (along with new data if available).
Return code	
Std_ReturnType	> E_OK The request has been accepted > E_NOT_OK The request has not been accepted
Functional Description	
Transmit TCP message This service requests transmission of data via TCP to a remote node. The transmission of the data is decoupled.	
Particularities and Limitations	
The TCP segment(s) are sent dependent on runtime factors (e.g. receive window) and configuration parameter (e.g. Nagle algorithm).	
Call context	
> task level	

Table 4-20 TcpIp_TcpTransmit

4.2.12 TcpIp_TcpReceived

Prototype	
`Std_ReturnType` **`TcpIp_TcpReceived`** `(TcpIp_SocketIdType SocketId, uint32 Length)`	
Parameter	
SocketId [in]	Socket identifier of the related local socket resource
Length [in]	Number of bytes finally consumed by the upper layer
Return code	
Std_ReturnType	> E_OK The request has been accepted > E_NOT_OK The request has not been accepted
Functional Description	
By this API service the reception of socket data is confirmed to the TCP/IP stack.	
Particularities and Limitations	
Increase the TCP receive window of the socket specified by SocketId considering the number of finally consumed bytes specified by Length.	
Call context	
> interrupt or task level	

Table 4-21 TcpIp_TcpReceived

4.2.13 TcpIp_Close

Prototype	
`Std_ReturnType TcpIp_Close (TcpIp_SocketIdType SocketId, boolean Abort)`	
Parameter	
SocketId [in]	Socket handle identifying the local socket resource.
Abort [in]	TRUE: connection will immediately be terminated by sending a RST-Segment and releasing all related resources. FALSE: connection will be terminated after performing a regular connection termination handshake and releasing all related resources.
Return code	
Std_ReturnType	> E_OK The request has been accepted. > E_NOT_OK The request has not been accepted.
Functional Description	
Close the socket. By this API service the TCP/IP stack is requested to close the socket and release all related resources. The service TcpIp_Close() shall perform the following actions for the socket specified by SocketId in case it is a TCP socket: (a) if the connection is active and (a1) abort = FALSE: the connection shall be terminated after performing a regular connection termination handshake and releasing all related resources. (a2) abort = TRUE: connection shall immediately be terminated by sending a RST-Segment and releasing all related resources. (b) if the socket is in the Listen state, the Listen state shall be left immediately and related resources shall be released.	
Particularities and Limitations	
Call context	
> task level	

Table 4-22 TcpIp_Close

4.2.14 TcpIp_IcmpV6Transmit

Prototype	
`Std_ReturnType TcpIp_IcmpV6Transmit(TcpIp_LocalAddrIdType LocalIpAddrId, const TcpIp_SockAddrType *RemoteAddrPtr, uint8 HopLimit, uint8 Type, uint8 Code, uint16 DataLength, const uint8 *DataPtr)`	
Parameter	
LocalIpAddrId [in]	Local address identifier. (must reference an IPv6 address)
RemoteAddrPtr [in]	Destination address. (must be an IPv6 address)
HopLimit [in]	HopLimit value of the outgoing IPv6 packet.
Type [in]	Value of the Type field in the ICMPv6 message header.
Code [in]	Value of the Code field in the ICMPv6 message header.
DataLength [in]	Length of the message payload.
DataPtr [in]	Message payload.
Return code	
TcpIp_ReturnType	> E_OK Message was transmitted. > E_NOT_OK Message was not transmitted due to pending link-layer address resolution.
Functional Description	
Sends a raw ICMPv6 message of specified Type and Code.	
Particularities and Limitations	
This function may also be used to send ICMPv6 Echo Requests. Replies to send ICMPv6 messages can be received via the <Up>_IcmpMsgHandler callback.	
Call context	
> task level	

Table 4-23 TcpIp_IcmpV6Transmit

4.2.15 TcpIp_RequestIpAddrAssignment

Prototype	
`Std_ReturnType` **`TcpIp_RequestIpAddrAssignment`** `(TcpIp_LocalAddrIdType LocalAddrId, TcpIp_IpAddrAssignmentType Type, TcpIp_SockAddrType *LocalIpAddrPtr, uint8 Netmask, TcpIp_SockAddrType *DefaultRouterPtr)`	
Parameter	
LocalAddrId [in]	IP address index specifying the IP address for which an assignment shall be initiated.
Type [in]	type of IP address assignment which shall be initiated
LocalIpAddrPtr [in]	pointer to structure containing the IP address which shall be assigned to the EthIf controller indirectly specified via LocalAddrId. Note: This parameter is only used in case the parameters Type is set to TCPIP_IPADDR_ASSIGNMENT_STATIC.
Netmask [in]	Network mask of IPv4 address or address prefix of IPv6 address in CIDR Notation.
DefaultRouterPtr [in]	Pointer to struct where the IP address of the default router (gateway) is stored (struct member 'port' is not used and of arbitrary value) (IPv4 only)
Return code	
Std_ReturnType	> E_OK The request has been accepted > E_NOT_OK The request has not been accepted
Functional Description	
Initiate the local IP address assignment for the IP address specified by LocalAddrId.	
Particularities and Limitations	
Call context	
> task level	

Table 4-24 TcpIp_RequestIpAddrAssignment

4.2.16 TcpIp_ReleaseIpAddrAssignment

Prototype	
`Std_ReturnType` **`TcpIp_ReleaseIpAddrAssignment`** `(TcpIp_LocalAddrIdType LocalAddrId)`	
Parameter	
LocalAddrId [in]	IP address index specifying the IP address for which an assignment shall be released.
Return code	
Std_ReturnType	> E_OK The request has been accepted > E_NOT_OK The request has not been accepted
Functional Description	
Release the local IP address assignment for the IP address specified by LocalAddrId.	
Particularities and Limitations	
Call context	
> task level	

Table 4-25 TcpIp_ReleaseIpAddrAssignment

4.2.17 TcpIp_ReleaseSpecificIpAddrAssignment

Prototype	
`Std_ReturnType` **`TcpIp_ReleaseIpAddrAssignment`** `(TcpIp_LocalAddrIdType LocalAddrId, TcpIp_IpAddrAssignmentType AssignmentType)`	
Parameter	
LocalAddrId [in]	IP address index specifying the IP address for which an assignment shall be released.
AssignmentType [in]	Type of the assignment that shall be released: TCPIP_IPADDR_ASSIGNMENT_STATIC TCPIP_IPADDR_ASSIGNMENT_DHCP TCPIP_IPADDR_ASSIGNMENT_LINKLOCAL
Return code	
Std_ReturnType	> E_OK The request has been accepted > E_NOT_OK The request has not been accepted
Functional Description	
This API may be used if multiple IP address assignment methods are configured for an IPv4 address and only a specific address assignment shall be released. If all assignments for a local address shall be released use TcpIp_ReleaseIpAddrAssignment().	
Particularities and Limitations	
Call context	
> task level	

Table 4-26 TcpIp_ReleaseIpAddrAssignment

4.2.18 TcpIp_GetCtrlIdx

Prototype	
`Std_ReturnType TcpIp_GetCtrlIdx (TcpIp_LocalAddrIdType LocalAddrId, uint8 *CtrlIdxPtr)`	
Parameter	
LocalAddrId [in]	Local address identifier implicitely specifing the EthIf controller that shall be returned
CtrlIdxPtr [out]	Pointer to the memory where the index of the EthIf controller related to LocalAddrId is stored
Return code	
Std_ReturnType	> E_OK The request was successful > E_NOT_OK The request was not successful
Functional Description	
TcpIp_GetCtrlIdx returns the index of the EthIf controller related to LocalAddrId.	
Particularities and Limitations	
Call context	
> task level	

Table 4-27 TcpIp_GetCtrlIdx

4.2.19 TcpIp_GetIpAddr

Prototype	
`Std_ReturnType` **`TcpIp_GetIpAddr`** `(TcpIp_LocalAddrIdType LocalAddrId, TCPIP_P2V(TcpIp_SockAddrType) IpAddrPtr, TCPIP_P2V(uint8) NetmaskPtr, TCPIP_P2V(TcpIp_SockAddrType) DefaultRouterPtr)`	
Parameter	
LocalAddrId [in]	local address identifier
IpAddrPtr [out]	Pointer to a struct where the IP address is stored. (only struct members family and address will be updated)
NetmaskPtr [out]	Pointer to memory where Network mask of IPv4 address or address prefix of IPv6 address in CIDR notation is stored.
DefaultRouterPtr [out]	Pointer to struct where the IP address of the default router (gateway) is stored (only struct members family and address will be updated)
Return code	
Std_ReturnType	> E_OK The request was successful > E_NOT_OK The request was not successful.
Functional Description	
Obtains the local IP address actually used by LocalAddrId, the netmask and default router.	
Particularities and Limitations	
The output parameters IpAddrPtr, NetmaskPtr, DefaultRouterPtr may be set to NULL_PTR if the information is not required.	
Call context	
> task level	

Table 4-28 TcpIp_GetIpAddr

4.2.20 TcpIp_GetIpAddrCfgSrc

Prototype	
`Std_ReturnType` **`TcpIp_GetIpAddrCfgSrc`** `(TcpIp_LocalAddrIdType LocalAddrId, uint8 *CfgSrcPtr)`	
Parameter	
LocalAddrId [in]	local address identifier
AddrPtr [in]	address for which the configuration source shall be returned
CfgSrcPtr [out]	configuration source of the address
Return code	
Std_ReturnType	> E_OK address is valid and *CfgSrcPtr has be set > E_NOT_OK address invalid or not assigned to this node. *CfgSrcPtr not modified.
Functional Description	
Get the configuration source of an IP address.	
Particularities and Limitations	
Call context	
> task level	

Table 4-29 TcpIp_GetIpAddrCfgSrc

4.2.21 TcpIp_GetRemNetAddr

Prototype	
`Std_ReturnType` **`TcpIp_GetRemNetAddr`** `(TcpIp_SockHndType SockHnd, TCPIP_P2V(TcpIp_SockAddrType)*SockAddrPtr)`	
Parameter	
SockHnd [in]	socket handle
SockAddrPtr [out]	Pointer to a socket address element
Return code	
Std_ReturnType	> E_OK reading the socket address was successful > E_NOT_OK remote IP address could not be obtained
Functional Description	
The API provides the remote address of an established TCP socket connection.	
Particularities and Limitations	
SockAddrPtr is a pointer to a TcpIp memory area where the remote address is stored. The pointer is valid until the socket is closed.	
Call context	
> task level	

Table 4-30 TcpIp_GetRemNetAddr

4.2.22 TcpIp_GetLocSockAddr

Prototype	
`Std_ReturnType` **`TcpIp_GetLocSockAddr`** `(TcpIp_SockHndType SockHnd, TcpIp_SockAddrType *SockPtr)`	
Parameter	
SockHnd [in]	socket handle
SockPtr [out]	pointer were the local socket address shall be written to
Return code	
Std_ReturnType	> E_OK socket address was written successfully > E_NOT_OK error, socket address is not updated
Functional Description	
Read the local socket address of one socket.	
Particularities and Limitations	
: This is a Vector extension	
Call context	
> task level	

Table 4-31 TcpIp_GetLocSockAddr

4.2.23 TcpIp_GetPhysAddr

Prototype	
`Std_ReturnType` **`TcpIp_GetPhysAddr`** `(TcpIp_LocalAddrIdType LocalAddrId, uint8 *PhysAddrPtr)`	
Parameter	
LocalAddrId [in]	Local address identifier implicitly specifying the EthIf controller for which the physical address shall be obtained.
PhysAddrPtr [out]	Pointer to the memory where the physical source address (MAC address) in network byte order is stored
Return code	
Std_ReturnType	> E_OK The request was successful > E_NOT_OK The request was not successful, e.g. no unique Ctrl specified via IpAddrId.
Functional Description	
Obtains the physical source address used by the EthIf controller implicitly specified via LocalAddrId.	
Particularities and Limitations	
Call context	
> task level	

Table 4-32 TcpIp_GetPhysAddr

4.2.24 TcpIp_GetRemotePhysAddr

Prototype	
`TcpIp_ReturnType` **`TcpIp_GetRemotePhysAddr`** `(uint8 CtrlIdx, TcpIp_SockAddrType *IpAddrPtr, uint8 *PhysAddrPtr, boolean initRes)`	
Parameter	
CtrlIdx [in]	EthIf controller index to identify the related ARP/NDP table.
IpAddrPtr [in]	specifies the IP address for which the physical address shall be retrieved
initRes [in]	specifies if the address resolution shall be initiated (TRUE) or not (FALSE) in case the physical address related to the specified IP address is currently unknown.
PhysAddrPtr [out]	Pointer to the memory where the physical address (MAC address) related to the specified IP address is stored in network byte order.
Return code	
TcpIp_ReturnType	> TCPIP_E_OK specified IP address resolved, physical address provided via PhysAddrPtr > TCPIP_E_PHYS_ADDR_MISS physical address currently unknown (address resolution initiated if initRes set to TRUE)
Functional Description	
Lookup the physical address for a remote network address. TcpIp_GetRemotePhysAddr queries the IP/physical address translation table specified by CtrlIdx and returns the physical address related to the IP address specified by IpAddrPtr. In case no physical address can be retrieved and parameter initRes is TRUE, address resolution for the specified IP address is initiated on the local network.	
Particularities and Limitations	
Call context	
> task level	

Table 4-33 TcpIp_GetRemotePhysAddr

4.2.25 TcpIp_Tls_SetClientCertInfo

Prototype	
`Std_ReturnType TcpIp_Tls_SetClientCertInfo (TcpIp_SocketIdType SocketId, uint8 *CertPtr, uint8 *KeyPtr, uint16 CertLen, uint16 KeyLen)`	
Parameter	
SocketId [in]	socket handle
CertPtr [in]	pointer to client certificate (RAM block)
KeyPtr [in]	pointer to client private key (RAM block)
CertLen [in]	length of client certigicate
KeyLen [in]	length of client private key
Return code	
Std_ReturnType	> E_OK cert info could be set > E_NOT_OK cert info could NOT be set
Functional Description	
Set the client certificate and key block ids for a TLS connection (necessary for client authentication).	
Particularities and Limitations	
Call context	
> task level	

Table 4-34 TcpIp_Tls_SetClientCertInfo

4.2.26 TcpIp_Tls_GetNvmBlockIdForUsedRootCert

Prototype	
`Std_ReturnType TcpIp_Tls_GetNvmBlockIdForUsedRootCert (TcpIp_SocketIdType SocketId, NvM_BlockIdType *RootCertBlockIdPtr)`	
Parameter	
SocketId [in]	Socket id
RootCertBlockIdPtr [out]	Pointer to the block ID
Return code	
Std_ReturnType	> E_OK Block ID can be provided > E_NOT_OK request failed
Functional Description	
Get the NVM block ID of the root certificate used for a TLS connection.	
Particularities and Limitations	
Call context	
> task level	

Table 4-35 TcpIp_Tls_GetNvmBlockIdForUsedRootCert

4.2.27 TcpIp_Tls_RootCertWasModified

Prototype	
`Std_ReturnType` **`TcpIp_Tls_RootCertWasModified`** `(NvM_BlockIdType NvmBlockId)`	
Parameter	
NvmBlockId [in]	BlockId of the NVM block that has changed
Return code	
Std_ReturnType	> E_OK Block is used as a root certificate > E_NOT_OK request failed, block is not used as a root certificate
Functional Description	
Inform the TLS that a root cert has been updated in NVM.	
Particularities and Limitations	
Call context	
> task level	

Table 4-36 TcpIp_Tls_RootCertWasModified

4.2.28 TcpIp_DhcpReadOption

Prototype	
`Std_ReturnType` **`TcpIp_DhcpReadOption`** `(TcpIp_LocalAddrIdType LocalIpAddrId, uint8 Option, uint8 *DataLength, uint8 *DataPtr)`	
Parameter	
LocalIpAddrId [in]	IP address identifier representing the local IP address and EthIf controller for which the DHCP option shall be read.
Option [in]	DHCP option, e.g. Host Name
DataLength [in, out]	As input parameter, contains the length of the provided data buffer. Will be overwritten with the length of the actual data.
DataPtr [out]	Pointer to memory containing DHCP option data.
Return code	
Std_ReturnType	> E_OK requested data retrieved successfully. > E_NOT_OK requested data could not be retrieved.
Functional Description	
Read user option data obtained from incoming DHCPv4 messages. By this API service the TCP/IP stack retrieves DHCP option data identified by parameter option for already received DHCP options.	
Particularities and Limitations	
The API can only be used to read out the content of statically configured DHCP options.	
Call context	
> interrupt or task level	

Table 4-37 TcpIp_DhcpReadOption

4.2.29 TcpIp_DhcpWriteOption

Prototype	
`Std_ReturnType` **`TcpIp_DhcpWriteOption`** `(TcpIp_LocalAddrIdType LocalIpAddrId, uint8 Option, uint8 DataLength, const uint8 *DataPtr)`	
Parameter	
LocalIpAddrId [in]	IP address identifier representing the local IP address and EthIf controller for which the DHCP option shall be written.
Option [in]	DHCP option, e.g. Host Name
DataLength [in]	Length of DHCP option data in bytes. Set to 0 to clear previously stored data
DataPtr [in]	Pointer to memory containing DHCP option data
Return code	
Std_ReturnType	> E_OK no error occurred. > E_NOT_OK DHCP option data could not be written.
Functional Description	
Set user option data for outgoing DHCPv4 messages. Invoking the API with DataLength as 0 will clear the user option data previously stored. By this API service the TCP/IP stack writes the DHCP option data identified by parameter option.	
Particularities and Limitations	
Old similar Vector API was TcpIp_SetDhcpV4TxOption. The API can only be used to write the content of statically configured DHCP options.	
Call context	
> interrupt or task level	

Table 4-38 TcpIp_DhcpWriteOption

4.2.30 TcpIp_DhcpV6ReadOption

Prototype	
`Std_ReturnType` **`TcpIp_DhcpV6ReadOption`** `(TcpIp_LocalAddrIdType LocalIpAddrId, uint16 Option, uint16 *DataLength, uint8 *DataPtr)`	
Parameter	
LocalIpAddrId [in]	IP address identifier representing the local IP address and EthIf controller for which the DHCP option shall be read.
Option [in]	DHCP option, e.g. Host Name
DataLength [in,out]	As input parameter, contains the length of the provided data buffer. Will be overwritten with the length of the actual data.
DataPtr [out]	Pointer to memory containing DHCPv6 option data.
Return code	
Std_ReturnType	> E_OK requested data retrieved successfully. > E_NOT_OK requested data could not be retrieved.
Functional Description	
Read user option data obtained from incoming DHCPv6 messages. By this API service the TCP/IP stack retrieves DHCP option data identified by parameter option for already received DHCP options.	
Particularities and Limitations	
The API can only be used to read out the content of statically configured DHCPv6 options.	
Call context	
> interrupt or task level	

Table 4-39 TcpIp_DhcpV6ReadOption

4.2.31 TcpIp_DhcpV6WriteOption

Prototype	
`Std_ReturnType` **`TcpIp_DhcpV6WriteOption`** `(TcpIp_LocalAddrIdType LocalIpAddrId, uint16 Option, uint16 DataLength, const uint8 *DataPtr)`	
Parameter	
LocalIpAddrId [in]	IP address identifier representing the local IP address and EthIf controller for which the DHCP option shall be read.
Option [in]	DHCP option, e.g. Host Name
DataLength [in]	Length of option data in bytes. Set to 0 to clear previously stored data
DataPtr [in]	Payload of the option. Pointer must be valid until it is updated by calling this function. (Ignored if DataLength is 0)
Return code	
Std_ReturnType	> E_OK No error occurred > E_NOT_OK DHCP option data could not be written
Functional Description	
Set user option data for outgoing DHCPv6 messages. Invoking the API with DataLength as 0 will clear the option data previously stored.	
Particularities and Limitations	
Old similar Vector API was TcpIp_SetDhcpV6TxOption. The API can only be used to write the content of statically configured DHCPv6 options.	
Call context	
> interrupt or task level	

Table 4-40 TcpIp_DhcpV6WriteOption

4.2.32 TcpIp_GetDhcpTimeoutInfo

Prototype	
`boolean TcpIp_GetDhcpTimeoutInfo (uint8 CtrlIdx, uint8 IpVer)`	
Parameter	
EthIfCtrlIdx [in]	controller index
IpVer [in]	IP version
Return code	
boolean	> TRUE there was a DHCP timeout > FALSE there was no DHCP timeout
Functional Description	
check if a DHCP address finding timeout occurred	
Particularities and Limitations	
Call context	
> interrupt or task level	

Table 4-41 TcpIp_GetDhcpTimeoutInfo

4.2.33 TcpIp_DhcpV4_GetStatus

Prototype	
`Std_ReturnType` **`TcpIp_DhcpV4_GetStatus`**`( IpV4_AddrIdType IpAddrId, uint8* DhcpStatePtr )`	
Parameter	
IpAddrId [in]	local address identifier
DhcpStatePtr [out]	current, simplified, DHCP status of the IP address
Return code	
Std_ReturnType	> E_OK the DhcpState is valid > E_NOT_OK given address is invalid or has no DHCP configured or the IP controller of the given address is not in state online active,
Functional Description	
Returns the current DCHP state of a given IP address. The following simplified DHCP states are supported: TCPIP_DHCP_SIMPLIFIED_STATE_NOT_ACTIVE: DHCP client not active. TCPIP_DHCP_SIMPLIFIED_STATE_NO_DISCVR_SENT: No discover sent. TCPIP_DHCP_SIMPLIFIED_STATE_DISCVR_SENT_NO_ANSWR_RCVD_YET: Discover sent but no answer received. TCPIP_DHCP_SIMPLIFIED_STATE_OFFER_RCVD_WITHOUT_REQ: DHCP offer received, no request sent. TCPIP_DHCP_SIMPLIFIED_STATE_REQ_SENT_NO_ACK_RCVD_YET: DHCP request sent, no ACK received. TCPIP_DHCP_SIMPLIFIED_STATE_ACK_RCVD_DHCP_ADDR_ASSIGND: DHCP address assigned to current interface.	
Particularities and Limitations	
This is a Vector extension.	
Call context	
> task level	

Table 4-42 TcpIp_DhcpV4_GetStatus

4.2.34 TcpIp_RequestComMode

Prototype	
`Std_ReturnType TcpIp_RequestComMode (uint8 CtrlIdx, TcpIp_StateType State)`	
Parameter	
CtrlIdx [in]	EthIf controller index to identify the communication network where the TcpIp state is requested
State [in]	Requested TcpIp state
Return code	
Std_ReturnType	> E_OK request was accepted > E_NOT_OK request was not accepted
Functional Description	
By this API service the TCP/IP stack is requested to change the TcpIp state of the communication network identified by EthIf controller index.	
Particularities and Limitations	
Currently the request of the following states is supported: - TCPIP_STATE_ONLINE - TCPIP_STATE_OFFLINE The request of the TCPIP_STATE_ONHOLD leads to the same behavior as the request of the state OFFLINE.	
Call context	
> task level	

Table 4-43 TcpIp_RequestComMode

4.2.35 TcpIp_DiagDataReadAccess

Prototype	
`Std_ReturnType` **`TcpIp_DiagDataReadAccess`** `(TcpIp_SockHndType SockHnd, TcpIp_DiagParamsType DataID, uint8 *DataPtr, uint16 *DataLenPtr)`	
Parameter	
SockHnd [in]	socket handle
DataID [in]	data identifier
DataPtr [out]	pointer for diagnostic data
DataLenPtr [in,out]	pointer for maximum / actual length of diagnostic data in bytes
Return code	
Std_ReturnType	> E_OK diagnostic data written > E_NOT_OK invalid parameter (data identifier not found, NULL_PTR parameter, invalid length)
Functional Description	
diagnostic data read access	
Particularities and Limitations	
The memory fragment DataPtr points to should be aligned properly regarding the expected returned type / struct. Data is only written if RetValue is E_OK. This is a Vector extension.	
Call context	
> task level	

Table 4-44 TcpIp_DiagDataReadAccess

4.2.36 TcpIp_ClearARCache

Prototype	
`Std_ReturnType` **`TcpIp_ClearARCache`** `(TcpIp_LocalAddrIdType LocalAddrId)`	
Parameter	
LocalAddrId [in]	Local address identifier implicitly specifying the IPv4/IPv6 controller that shall be cleared.
Return code	
Std_ReturnType	> E_OK The request was successful > E_NOT_OK The request was not successful
Functional Description	
TcpIp_ClearARCache clears the address resolution cache.	
Particularities and Limitations	
Currently only IPv6/NDP is supported	
Call context	
> task level	

Table 4-45 TcpIp_ClearARCache

4.2.37 TcpIp_GetArpCacheEntries

Prototype	
`Std_ReturnType` **`TcpIp_GetArpCacheEntries`** `(uint8 ctrlIdx, uint32 *numberOfElements, TcpIp_ArpCacheEntryType *entryListPtr)`	
Parameter	
ctrlIdx [in]	EthIf controller index to identify the related ARP table.
numberOfElements[in,out]	In: Maximum number of entries that can be stored in the output buffer entryListPtr. Out: Number of entries that are written into the output buffer entryListPtr (Total number of all entries in the cache if input value is 0).
entryListPtr[out]	Pointer to memory where the list of cache entries shall be stored. if *numberOfElements is 0 , the output may be NULL_PTR
Return code	
Std_ReturnType	> E_OK physical address cache could be read. > E_NOT_OK physical address cache could not be read. (i.e. no IPv4 instance active on this controller)
Functional Description	
Returns entries that are currently stored in the IPv4 address resolution cache. Copies entries from the physical address cache of the IPv4 instance that is active on the EthIf controller specified by ctrlIdx into a user provided buffer. The function will copy all or numberOfElements into the output list. If input value of numberOfElements is 0 the function will not copy any data but only return the number of entries in the cache. entryListPtr may be NULL_PTR in this case.	
Particularities and Limitations	
Call context	
> interrupt or task level	

Table 4-46 TcpIp_GetArpCacheEntries

4.2.38 TcpIp_GetNdpCacheEntries

Prototype	
`Std_ReturnType` **`TcpIp_GetNdpCacheEntries`** `(uint8 ctrlIdx, uint32 *numberOfElements, TcpIp_NdpCacheEntryType *entryListPtr)`	
Parameter	
ctrlIdx [in]	EthIf controller index to identify the related NDP table.
numberOfElements[in,out]	In: Maximum number of entries that can be stored in Parameters output entryListPtr. Out: Number of entries written to output entryListPtr (Number of all entries in the cache if input value is 0).
entryListPtr[out]	Pointer to memory where the list of cache entries shall be stored. May only be NULL_PTR if *numberOfElements is 0.
Return code	
Std_ReturnType	> E_OK physical address cache could be read. > E_NOT_OK physical address cache could not be read. (i.e. no IPv6 instance active on this controller)
Functional Description	
Returns entries that are currently stored in the IPv6 link layer address resolution cache. Copies entries from the physical address cache of the IPv6 instance that is active on the EthIf controller specified by ctrlIdx into a user provided buffer. The function will copy all or numberOfElements into the output list. If input value of numberOfElements is 0 the function will not copy any data but only return the number of entries in the cache. EntryListPtr may be NULL_PTR in this case.	
Particularities and Limitations	
Call context	
> interrupt or task level	

Table 4-47 TcpIp_GetNdpCacheEntries

4.2.39 TcpIp_SendGratuitousArpReq

Prototype	
`Std_ReturnType` **`TcpIp_SendGratuitousArpReq`**`(TcpIp_LocalAddrIdType LocalAddrId)`	
Parameter	
LocalAddrId [in]	Local address identifier implicitly specifing the EthIf controller for which the physical address shall be obtained.
Return code	
Std_ReturnType	> E_OK the gratuitous ARP request was sent using the related lower layer APIs. > E_NOT_OK otherwise.
Functional Description	
Sends a gratuitous ARP request packet as specified in [RFC2002 4.6. second indentation]	
Particularities and Limitations	
This is a Vector extension.	
Call context	
> task level	

Table 4-48 TcpIp _SendGratuitousArpReq

4.2.40 IpV4_GetLastDuplicateDhcpAddrDid

Prototype	
`FUNC(Std_ReturnType, IPV4_CODE) IpV4_GetLastDuplicateDhcpAddrDid(uint8 IpCtrlIdx, CONSTP2VAR(IpBase_AddrInType, AUTOMATIC, IPV4_APPL_DATA) IpAddrPtr, CONSTP2VAR(IpBase_EthPhysAddrType, AUTOMATIC, IPV4_APPL_DATA) PhysAddrPtr)`	
Parameter	
IpCtrlIdx [in]	Ip controller index
IpAddrPtr [out]	Specifies the memory where the IP address shall be stored for which a duplicated DHCP address assignment has been detected.
PhysAddrPtr [out]	Specifies the memory where the physical address shall be stored for which a duplicated DHCP address assignment has been detected.
Return code	
Std_ReturnType	> E_OK the given IpCtrlIdx was valid and the DID information was stored in the given data structures. > E_NOT_OK otherwise.
Functional Description	
Returns the DID for DEM event TCPIP_E_DUPLICATE_DHCP_ADDR	
Particularities and Limitations	
This is a Vector extension.	
Call context	
> task level	

Table 4-49 IpV4_GetLastDuplicateDhcpAddrDid

4.2.41 TcpIp_MainFunction

Prototype	
`void TcpIp_MainFunction (void)`	
Parameter	
void [in]	none
Return code	
void	void
Functional Description	
The TcpIp MainFunction handles the module internal state handling. Calls all TcpIp-internal MainFunction functions. This function has to be called cyclically.	
Particularities and Limitations	
Init has to be called before	
Call context	
> task level	

Table 4-50 TcpIp_MainFunction

4.2.42 TcpIp_MainFunctionRx

Prototype	
`void TcpIp_MainFunctionRx (void)`	
Parameter	
void [in]	none
Return code	
void	void
Functional Description	
The TcpIp MainFunctionRx handles all received data and forwards information to upper layers. In case this function is called, the TcpIp_MainFunctionTx and TcpIp_MainFunctionState HAVE TO BE called as well and the TcpIp_MainFunction MUST NOT be called.	
Particularities and Limitations	
Init has to be called before	
Call context	
> task level	

Table 4-51 TcpIp_MainFunctionRx

4.2.43 TcpIp_MainFunctionTx

Prototype	
`void TcpIp_MainFunctionTx (void)`	
Parameter	
void [in]	none
Return code	
void	void
Functional Description	
The TcpIp MainFunctionTx handles all pending transmit requests from the upper layers and transmits the data. In case this function is called, the TcpIp_MainFunctionRx and TcpIp_MainFunctionState HAVE TO BE called as well and the TcpIp_MainFunction MUST NOT be called.	
Particularities and Limitations	
Init has to be called before	
Call context	
> task level	

Table 4-52 TcpIp_MainFunctionTx

4.2.44 TcpIp_MainFunctionState

Prototype	
`void TcpIp_MainFunctionState (void)`	
Parameter	
void [in]	none
Return code	
void	void
Functional Description	
The TcpIp MainFunction handles the module internal state handling and timing actions. In case this function is called, the TcpIp_MainFunctionRx and TcpIp_MainFunctionTx HAVE TO BE called as well and the TcpIp_MainFunction MUST NOT be called.	
Particularities and Limitations	
Init has to be called before	
Call context	
> task level	

Table 4-53 TcpIp_MainFunctionState

4.2.45 TcpIp_GetLocNetAddr

Prototype	
`Std_ReturnType TcpIp_GetLocNetAddr (TcpIp_LocalAddrIdType LocalAddrId, TcpIp_NetAddrType *NetAddrPtr)`	
Parameter	
LocalAddrId [in]	local address identifier
NetAddrPtr [out]	pointer for the local network address
Return code	
Std_ReturnType	> E_OK local network address returned > E_NOT_OK local network address access failed
Functional Description	
This function returns the current IP address for a given controller.	
Particularities and Limitations	
This API is deprecated and will be removed in future revisions of this module.	
Call context	
> task level	

Table 4-54 TcpIp_GetLocNetAddr

4.2.46 TcpIp_GetLocNetMask

Prototype	
`Std_ReturnType **TcpIp_GetLocNetMask** (TcpIp_LocalAddrIdType LocalAddrId, TcpIp_NetAddrType *NetMaskPtr)`	
Parameter	
LocalAddrId [in]	local address identifier
NetMaskPtr [out]	pointer for the local network mask
Return code	
Std_ReturnType	> E_OK local network mask returned > E_NOT_OK local network mask access failed
Functional Description	
This function returns the current network mask for a given controller.	
Particularities and Limitations	
This API is deprecated and will be removed in future revisions of this module.	
Call context	
> task level	

Table 4-55 TcpIp_GetLocNetMask

4.2.47 TcpIp_GetAndResetMeasurementData

Prototype	
`Std_ReturnType` **`TcpIp_GetAndResetMeasurementData`** `(TcpIp_MeasurementIdxType MeasurementIdx, boolean MeasurementResetNeeded, uint32* MeasurementDataPtr)`	
Parameter	
`MeasurementIdx` [in]	Data index of measurement data
`MeasurementResetNeeded` [in]	Flag to trigger a reset of the measurement data
`MeasurementDataPtr` [out]	Reference to data buffer, where to copy measurement data
Return code	
Std_ReturnType	> E_OK successful > E_NOT_OK failed
Functional Description	
This function allows to read and reset detailed measurement data for diagnostic purpose. It allows to read the number of dropped TCP/UDP packets, because those packets could not be mapped to valid destination (Local) Ip and Port. Additionally, the function allows to read the number of TCP/UDP packets, dropped due to Invalid header or Invalid checksum. It allows to read the number of IPV4/IPV6 packet, dropped due to invalid destination Ip address. Additionally, it also allows to read the number of IPV6 packets dropped, due to Incorrect packet format, Invalid extension header and Invalid codes and types for ICMPv6 packets. This function also allows to read the number of dropped TCP packets due to insufficient Tx/Rx buffer and the number of dropped IPV4 packets due to Insufficient reassembly buffer. If IPsec support is enabled, the API can be invoked to read the statuses of all the SA entry pairs. The vendor specific indices, TCPIP_MEAS_VENDOR_SPECIFIC_NR_SA_PAIRS and TCPIP_MEAS_VENDOR_SPECIFIC_SA_PAIRS can be used to read the number of SA entries and the statuses of SA entries present in the SAD, respectively. To read the IPsec statuses, the API MUST be called first with TCPIP_MEAS_VENDOR_SPECIFIC_NR_SA_PAIRS, and the second call with TCPIP_MEAS_VENDOR_SPECIFIC_SA_PAIRS and enough size of `MeasurementDataPtr` buffer. Get all Measurement Idxs at once is not supported. TCPIP_MEAS_ALL shall only be used to reset all Measurement Idxs (expect IPsec statuses) at once. Null pointer MUST be provided for `MeasurementDataPtr` in this case. Additionally, if API is called with parameter `MeasurementResetNeeded` set to TRUE and `MeasurementIdx` set to either TCP, UDP, IPV4 or IPV6 (but not TCPIP_MEAS_ALL), the respected measurement data will be first copied and then cleared. `MeasurementResetNeeded` parameter is ignored when called for IPsec status indices.	
Particularities and Limitations	
This API is only present, if configuration parameter TcpIp/TcpIpGeneral/TcpIpGetAndResetMeasurementDataApi == TRUE	
Call context	
> task level	

Table 4-56 TcpIp_GetAndResetMeasurementData



Caution
To invoke TcpIp_GetAndResetMeasurementData() for TCPIP_MEAS_VENDOR_SPECIFIC_SA_PAIRS, the `MeasurementDataPtr` buffer MUST have enough space to store the statuses of all valid SA entries. The exact size of the buffer is calculated as :

*Size = NumOfSaEntryPair * (sizeof(TcpIp_IpSecSaStatusType))*

where *NumOfSaEntryPair* is the value returned by reading TCPIP_MEAS_VENDOR_SPECIFIC_NR_SA_PAIRS

4.2.48 TcpIp_IpSecSaEntryPairAdd

Prototype	

`Std_ReturnType` **`TcpIp_IpSecSaEntryPairAdd`** `(`
`TcpIp_LocalAddrIdType LocalAddrId,`
`TCPIP_P2C(TcpIp_IpSecSaInfoType) SaInfoPtr)`	

Parameter	
LocalAddrId [in]	Local Address Id for which SA entry pair is created.
SaInfoPtr [in]	Pointer to struct of SA specific information. RemoteAddr field is expected to be in Nbo (inside SaSel). All port numbers are expected to be in Hbo (inside SaSel).
Return code	
Std_ReturnType	> E_OK SA Entry Pair has been added successfully. > E_NOT_OK SA Entry Pair could not be added successfully.
Functional Description	
This function Adds a pair of SA Entries for the controller referred by the Local Address Id.	
Particularities and Limitations	
IPsec is configured for the Controller	
Call context	
> task level	

Table 4-57 TcpIp_IpSecSaEntryPairAdd

4.2.49 TcpIp_IpSecSaEntryPairDelete

Prototype	
`Std_ReturnType` **`TcpIp_IpSecSaEntryPairDelete`**`(uint32 InboundSpi)`	
Parameter	
InboundSpi [in]	SPI of the inbound SA entry.
Return code	
Std_ReturnType	> E_OK SA Entry Pair has been deleted successfully. > E_NOT_OK SA Entry Pair could not be deleted.
Functional Description	
This function deletes a pair of SA Entries for the controller.	
Particularities and Limitations	
IPsec is configured for the Controller	
Call context	
> task level	

Table 4-58 TcpIp_IpSecSaEntryPairDelete

4.3 Services used by TCPIP

In the following table services provided by other components, which are used by the TCPIP are listed. For details about prototype and functionality refer to the documentation of the providing component.

Component	API
DET	Det_ReportError
EthIf	EthIf_ProvideTxBuffer
EthIf	EthIf_Transmit
EthIf	EthIf_GetPhysAddr
IpBase	IpBase_Copy
IpBase	IpBase_Fill
IpBase	IpBase_TcpIpChecksumAdd
IpBase	IpBase_DelSockAddr
IpBase	IpBase_CopySockAddr
IpBase	IpBase_SockPortIsEqual

Table 4-52 Services used by the TCPIP

4.4 Callback Functions

This chapter describes the callback functions that are implemented by the TCPIP and can be invoked by other modules. The prototypes of the callback functions are provided in the header file `TcpIp_Cbk.h` by the TCPIP.

4.4.1 TcpIp_RxIndication

Prototype	
`void TcpIp_RxIndication (uint8 CtrlIdx, Eth_FrameType FrameType, boolean IsBroadcast, uint8 *PhysAddrPtr, uint8 *DataPtr, uint16 LenByte)`	
Parameter	
CtrlIdx [in]	Index of the EthIf controller.
FrameType [in]	frame type of received Ethernet frame
IsBroadcast [in]	parameter to indicate a broadcast frame
PhysAddrPtr [in]	pointer to Physical source address (MAC address in network byte order) of received Ethernet frame
DataPtr [in]	Pointer to payload of the received Ethernet frame (i.e. Ethernet header is not provided).
LenByte [in]	Length of received data.
Return code	
void	void
Functional Description	
RxIndication from the EthIf. By this API service the TCP/IP stack gets an indication and the data of a received frame.	
Particularities and Limitations	
This function will simply forward the incoming RxIndication to the corresponding RxIndications in the modules IpV4 and IpV6.	
Call context	
> interrupt or task level	

Table 4-59 TcpIp_RxIndication

4.5 Configurable Interfaces

4.5.1 Notifications and Callouts

At its configurable interfaces the TCPIP defines notifications that can be mapped to callback functions provided by other modules. The mapping is not statically defined by the TCPIP but can be performed at configuration time. The function prototypes that can be used for the configuration have to match the appropriate function prototype signatures, which are described in the following sub-chapters.

4.5.1.1 <Up_IpV6MaxPayloadLenChanged>

Prototype	
`void` **`<Up_IpV6MaxPayloadLenChanged>`**`(uint8 CtrlIdx, const IpBase_SockAddrType *DstAddrPtr, uint16 Mtu)`	
Parameter	
`CtrlIdx`	EthIf controller index
`DstAddrPtr`	IPv6 address of the destination
`Mtu`	Maximum Transmission Unit (MTU) for the destination. This is the maximum payload length of the IPv6 packet in bytes. If `IpV6_ProvideTxBuffer()` is called with a `BufLen` larger than `Mtu` the packet will be automatically fragmented by the IpV6 if possible.
Return code	
`void`	-
Functional Description	
These callbacks will be called if an ICMPv6 Packet Too Big (Type 2) message has been received by the IpV6 and the message indicates that the MTU value is smaller than the currently used MTU value for a specific destination address.	
Particularities and Limitations	
> These callbacks will be called only if the PathMTU feature is enabled in the IpV6 configuration. > The name of this function is specified by the following configuration parameter: `TcpIp/TcpIpConfig/TcpIpIpConfig/TcpIpIpV6Config/TcpIpIcmpV6Config/TcpIpIcmpV6MsgHandler/TcpIpIcmpV6MaxPayloadLenChgHandlerName`	
Call context	
> interrupt or task context	

Table 4-60 Maximum Payload Length Change Callback

4.5.1.2 <Up_InvNdAddrListOptionHandler>

Prototype	
`void <Up_InvNdAddrListOptionHandler>(uint8 CtrlIdx, Eth_PhysAddrType *RemoteLLAddrPtr, IpV6_AddrType *AddrListPtr, uint8 AddrCount)`	
Parameter	
`CtrlIdx`	EthIf controller index
`RemoteLLAddrPtr`	Link-layer (physical) address of the remote node that send the address list.
`AddrListPtr`	List of IPv6 addresses of the remote node
`AddrCount`	Number of IPv6 addresses in the address list
Return code	
`void`	-
Functional Description	
These Callbacks will be called when an Inverse Neighbor Discovery Solicitation or Advertisement with an Address List Option has been received (see [RFC4443 3.2. Packet Too Big Message).	
Particularities and Limitations	
> These callbacks will be called only if the “Inverse Solicitations“ or “Inverse Advertisements” are enabled in the IpV6 configuration. > The callback will be called with an empty address list (`AddrListPtr == NULL_PTR && AddrCount == 0`) if there was no response to a user invoked Inverse Solicitation (see `IpV6_Ndp_SendInverseSolicitation()`). > The name of this function is specified by the following configuration parameter: `TcpIp/TcpIpConfig/TcpIpIpConfig/TcpIpIpV6Config/TcpIpIcmpV6Config/TcpIpIcmpV6MsgHandler/TcpIpIcmpV6IndAddrListReceivedHandlerName`	
Call context	
> interrupt or task context	

Table 4-61 Inverse Neighbor Discovery Address List Option Receive Callback

4.5.1.3 <Up_IcmpMsgHandler>

Prototype	
`void <Up_IcmpMsgHandler>(TcpIp_LocalAddrIdType LocalAddrId, const TcpIp_SockAddrType *RemoteAddrPtr, uint8 Ttl, uint8 Type, uint8 Code, uint16 DataLength, uint8 *DataPtr)`	
Parameter	
`LocalAddrId [in]`	Local address identifier representing the local IP address and EthIf controller where the ICMP message has been received
`RemoteAddrPtr [in]`	Pointer to struct representing the address of the ICMP sender.
`Ttl [in]`	Time to live value of the received ICMPv4 message.
`Type [in]`	Type field value of the received ICMP message (Note: the value of the type field determines the format of the remaining ICMP message data)
`Code [in]`	Code field value of the received ICMP message.
`DataLength [in]`	Length of ICMP message.
`DataPtr [in]`	Pointer to the received ICMP message.
Return code	
`void`	-
Functional Description	
By this API service the configured ICMP message handler function is called by the TCP/IP stack on reception of an ICMP message which is not handled by the TCP/IP stack.	
Particularities and Limitations	
> The name of this function is specified by the following configuration parameter: `TcpIp/TcpIpConfig/TcpIpIpConfig/TcpIpIpV4Config/TcpIpIcmpConfig/TcpIpIcmpMsgHandler/TcpIpIcmpMsgHandlerName`	
Call context	
> interrupt or task context	

Table 4-62 ICMP Destination Unreachable Message Callback

4.5.1.4 <Up_IcmpV6MsgHandler>

Prototype	
`void <Up_IcmpV6MsgHandler>(TcpIp_LocalAddrIdType LocalAddrId, const TcpIp_SockAddrType *RemoteAddrPtr, uint8 Ttl, uint8 Type, uint8 Code, uint16 DataLength, uint8 *DataPtr, uint16 MultiPartDataLength, uint8 *MultiPartDataPtr)`	
Parameter	
`LocalAddrId [in]`	Local address identifier representing the local IP address and EthIf controller where the ICMPv6 message has been received
`RemoteAddrPtr [in]`	Pointer to struct representing the address of the ICMPv6 sender.
`Ttl [in]`	IP Hop Limit value of the received ICMPv6 message.
`Type [in]`	Type field value of the received ICMPv6 message (Note: the value of the type field determines the format of the remaining ICMPv6 message data)
`Code [in]`	Code field value of the received ICMPv6 message.
`DataLength [in]`	Length of ICMPv6 message.
`DataPtr [in]`	Pointer to the received ICMPv6 message.
`MultiPartDataLength [in]`	Length of multi-part data in bytes.
`MultiPartDataPtr [in]`	ICMPv6 multi-part data (see [RFC4884 8. ICMP Extension Objects]).
Return code	
`void`	-
Functional Description	
By this API service the configured ICMPv6 message handler function is called by the TCP/IP stack on reception of an ICMPv6 message which is not handled by the TCP/IP stack. This implementation supports ICMPv6 multi-part messages according to IETF RFC 4884. If a message contains multi-part data MultiPartDataLength is > 0 and MultiPartDataPtr points to the multi-part data structure. The following ICMPv6 messages may contain multi-part data: - ICMPv6 Destination Unreachable (Type 1) (see [30] 3.1. Destination Unreachable Message). - ICMPv6 Time Exceeded message has been received (see [30] 3.3. Time Exceeded Message). If the message has the ICMPv6 Multi-Part message format `MultiPartExtPtr` points to the first ICMP Extension Object (see [35] 8. ICMP Extension Objects).	
Particularities and Limitations	
> `MultiPartExtLen` is the length of all extensions objects in bytes and may be 0. > The name of this function is specified by the following configuration parameter: `TcpIp/TcpIpConfig/TcpIpIpConfig/TcpIpIpV6Config/TcpIpIcmpV6Config/TcpIpIcmpV6MsgHandler/TcpIpIcmpV6MsgHandlerName`	
Call context	
> interrupt or task context	

Table 4-63 ICMPV6 Destination Unreachable Message Callback

4.5.1.5 <Up_LinkLocalAddrCandidateCallout>

Prototype	
`void <Up_LinkLocalAddrCandidateCallout> (TcpIp_LocalAddrIdType LocalAddrId, uint8 conflictCount, uint32 *addrCandidatePtr)`	
Parameter	
`LocalAddrId`	Identifier of the TcpIpLocalAddr that shall be configured using the LINKLOCAL address assignment method.
`conflictCount`	Number of conflicts that occurred since start of the address assignment for the local address. This value is 0 for the first probed address and is incremented every time a conflict has occurred.
`addrCandidatePtr`	Output parameter for the IP address candidate (IPv4 address in network-byte order). If this parameter is not changed inside the callout the IPv4 automatically generates a valid random IP address candidate.
Return code	
`void`	-
Functional Description	
This callout is called when a local unicast address shall be configured using the LINKLOCAL (Auto-IP) assignment method. The function is called every time a link-local address candidate is required. The callout is not required to modify *addrCandidatePtr if a random address according to IETF RFC 3927 shall be used by the IPv4.	
Particularities and Limitations	
> The returned address candidate must be in the range 169.254.1.0 to 169.254.254.255 (inclusive) in order to conform to IETF RFC 3927. The value is not checked by the IPv4. > The name of this function is specified by the following configuration parameter: `TcpIp/TcpIpConfig/TcpIpIpConfig/TcpIpIpV4Config/TcpIpAutoIpConfig/TcpIpAutoIpAddrCandidateCalloutFunction`	
Call context	
> interrupt or task context	

Table 4-64 IPv4 Link local Auto Ip address assignment Callback

4.5.1.6 <Up_PhysAddrTableChg>

Prototype	
`void <Up_PhysAddrTableChg>(uint8 CtrlIdx, TCPIP_P2V(TcpIp_SockAddrType) IpAddrPtr, TCPIP_P2V(uint8) PhysAddrPtr, boolean Valid);`	
Parameter	
`uint8 CtrlIdx`	EthIf controller index to identify the related ARP/NDP table.
`IpAddrPtr`	Specifies the memory where the IP address is stored for which the assignment to a physical address has changed.
`PhysAddrPtr`	Specifies the memory where the physical source address (MAC address) in network byte order is stored.
`Valid`	Flag, whether the couple of Ip address and physical address is valid or not.
Return code	
`void`	-
Functional Description	
This callback is called in order to notify upper layers about a physical address change.	
Particularities and Limitations	
> The name of this function is specified by the following configuration parameter: `TcpIp/TcpIpConfig/TcpIpPhysAddrConfig/TcpIpPhysAddrChgHandler/TcpIpPhysAddrChgHandlerName`	
Call context	
> interrupt or task context	

Table 4-65 Physical Address Change Callback

4.5.1.7 <Up_PhysAddrTableEntryDiscarded>

Prototype	
`void <Up_PhysAddrTableEntryDiscarded> (uint8 CtrlIdx, TCPIP_P2V(TcpIp_SockAddrType) IpAddrPtr);`	
Parameter	
`uint8 CtrlIdx`	EthIf controller index to identify the related ARP/NDP table.
`IpAddrPtr`	Specifies the memory where the IP address is stored for which the assignment to a physical address has changed.
Return code	
`Void`	-
Functional Description	
This callback is called in order to notify upper layers about the fact that a physical address change has been discarded because EcuC switch TcpIpArpDiscardedEntryHandling is true. Refer to section Tracking discarded ARP entries for details.	
Particularities and Limitations	
> The name of this function is specified by the following configuration parameter: `TcpIp/TcpIpConfig/TcpIpPhysAddrConfig/ TcpIpPhysAddrChgDiscardedHandler/TcpIpPhysAddrChgDiscardedHandlerName`	
Call context	
> interrupt or task context	

Table 4-66 Physical Address Change Discarded Callback

4.5.1.8 <Up_DADAddressConflict>

Prototype	
`void <Up_DADAddressConflict> (TcpIp_LocalAddrIdType IpAddrId,const TcpIp_SockAddrType* IpAddrPtr, const uint8* LocalPhysAddrPtr, const uint8* RemotePhysAddrPtr);`	
Parameter	
IpAddrId	local address identifier
IpAddrPtr	Specifies the memory where the IP address is stored for which a duplicate address has been detected.
LocalPhysAddrPtr	Specifies the memory where the local physical source address (MAC address) in network byte order is stored.
RemotePhysAddrPtr	Specifies the memory where the remote node's physical address (MAC address) in network byte order is stored.
Return code	
`Void`	-
Functional Description	
This callback is called in order to notify upper layers about the detection of a duplicate IP address.	
Particularities and Limitations	
> This callout is currently issued only in case duplicate address detection via NDP is done when IPv6 is active. > The name of this function is specified by the following configuration parameter: `TcpIp/TcpIpConfig/TcpIpDuplicateAddressDetectionConfig/ TcpIpDuplicateAddressDetectionCalloutName`	
Call context	
> interrupt or task context	

Table 4-67 Duplicate Address Detection Callback

4.5.1.9 <Up_DhcpRequestedIpAddrCallout>

Prototype	
`void <Up_DhcpRequestedIpAddrCallout> (TcpIp_LocalAddrIdType LocalAddrId, IpBase_AddrInType *RequestedAddrPtr)`	
Parameter	
`LocalAddrId`	Identifier of the TcpIpLocalAddr that shall be configured using the DHCP address assignment method.
`RequestedAddrPtr`	Input/Output parameter for the IP address that shall be requested in the DHCPDISCOVER message via the 'Requested IP Address' option (IPv4 address in network-byte order). If this parameter is not changed or set to TCPIP_INADDR_ANY inside the callout the DHCP will not request a particular IP address in the DHCPDISCOVER messages.
Return code	
`Void`	-
Functional Description	
This callout is called after the DHCP address assignment has been started but before the first DISCOVER message is sent by the DHCPv4 client. If an IP address is returned this address will be requested using the 'Requested IP Address' option in each DHCPDISCOVER message. If the DHCP server offers a different IP address the DHCP client will request the offered address and not the address requested in the DHCPDISCOVER message. According to IETF RFC 2131 "3.5 Client parameters in DHCP" the requested IP address in the DHCPDISCOVER message is only a suggestion of a particular IP address.	
Particularities and Limitations	
> The name of this function is specified by the following configuration parameter: `TcpIp/TcpIpConfig/TcpIpIpConfig/TcpIpIpV4Config/TcpIpDhcpConfig/TcpIpDhcpV4RequestedAddrCalloutFunction`	
Call context	
> interrupt or task context	

Table 4-68 DHCPv4 Requested IP address

4.5.1.10 <Up_DhcpEvent>

Prototype	
`void <Up_DhcpEvent> ( TcpIp_LocalAddrIdType LocalIpAddrId, TcpIp_DhcpEventType Event)`	
Parameter	
`LocalIpAddrId`	Identifier of the local IP address which received or which will transmit a DHCP message.
`Event`	Indicates the actual DHCP event. See “4.1 Type Definitions” for further details.
Return code	
`Void`	-
Functional Description	
This callout is only relevant for DHCPv4/DHCPv6 client functionality. The callout is invoked for the following events: > After reception of DHCPv4/DHCPv6 client message. > Prior transmission of DHCPv4/DHCPv6 client message. The callout can be used to synchronize the handling of DHCP options via the read and write APIs: • TcpIp_DhcpWriteOption • TcpIp_DhcpReadOption • TcpIp_DhcpV6WriteOption • TcpIp_DhcpV6ReadOption	
Particularities and Limitations	
> The name of this function is specified by the following configuration parameter: `TcpIp/TcpIpConfig/TcpIpSocketOwnerConfig/TcpIpSocketOwner/TcpIpSocketOwnerDhcpEventName`	
Call context	
> interrupt or task context	

Table 4-69 DHCP Event Callout

4.5.1.11 <Up_IpSecSpdCalloutFunction>

Prototype	
`void <Up_IpSecSpdCalloutFunction> ( TCPIP_P2V(TcpIp_IpSecPolicyType)SecurityPolicyPtr, uint8 Protocol, TcpIp_SockAddrBaseType RemSock, TcpIp_SockAddrBaseType LocSock)`	
Parameter	
`SecurityPolicyPtr`	Input/Output parameter for the Security policy which has been selected for the current IP packet processed (Tx or Rx). The values possible are PROTECT, BYPASS or OPTIONAL. This parameter can be overwritten by the caller. The caller is allowed to overwrite with only the following values BYPASS or DISCARD.
`Protocol`	The upper layer protocol in the IP packet.
`RemSock`	Remote IP address and port.
`LocSock`	Local IP address and port.
Return code	
`Void`	-
Functional Description	
This callout is called for each IP packet processed by the controller (Tx and Rx), after the Security Policy for the traffic has been looked up. The caller has the option of overwriting the security policy for the packet to either BYPASS in case they which to process the packet or DISCARD to drop the packet without further processing.	
Particularities and Limitations	
The name of this function is specified by the following configuration parameter: `TcpIp/TcpIpConfig/TcpIpIpSecConfigSet/TcpIpIpSecSpdCalloutFunction`	
Call context	
interrupt or task context	

Table 4-70 IPsec Security Policy Callout

4.5.1.12 <Up_IpSecAuditEventCalloutFunction>

Prototype	
`void <Up_ IpSecAuditEventCalloutFunction> ( uint32 Spi, uint32 SeqNum, TcpIp_SockAddrBaseType RemSock, TcpIp_SockAddrBaseType LocSock, TcpIp_IpSecEventType ErrorId)`	
Parameter	
`Spi`	SPI index of the audited packet.
`SeqNum`	Sequence Number of the audited packet.
`RemSock`	Remote IP address and port.
`LocSock`	Local IP address and port.
`ErrorId`	Auditable Error that is reported.
Return code	
`Void`	-
Functional Description	
This callout is invoked whenever an auditable event reported for the IPsec, to notify the upper layer.	
Particularities and Limitations	
The name of this function is specified by the following configuration parameter: `TcpIp/TcpIpConfig/TcpIpIpSecConfigSet/TcpIpIpSecAuditEventCalloutFunction`	
Call context	
interrupt or task context	

Table 4-71 IPsec Audit Event Callout

5 Configuration

The TCPIP is configured with the help of the configuration tool DaVinci Configurator Pro. The description of the parameters can be found in the BSWMD-file or in the properties window in the tool.

The following sections explain some general topics regarding the configuration.

5.1 Configuration Variants

The TCPIP supports the configuration variants

> VARIANT-PRE-COMPILE

The TCPIP supports the configuration of multiple variants (post-build variant support).

The configuration classes of the TCPIP parameters depend on the supported configuration variants. For their definitions please see the TcpIp_bswmd.arxml file.

5.2 Socket Owner Configuration

TCPIP allows multiple users above the TcpIp, and AUTOSAR calls them ‘socket owners’. The SocketAdaptor (SoAd) is the standard socket owner, other socket owners can be added easily as CDD (complex device driver). Vector furthermore adds a special socket owner ‘DhcpV4Server’ that is only used TCPIP-internal if the DhcpV4Server is enabled.

For each socket owner a number of callbacks have to be configured.

Callback Name	Required
<Up>_RxIndication	Mandatory
<Up>_TcpIpEvent	Mandatory
<Up>_TxConfirmation	Only required if UDP TX confirmation is requested.
<Up>_TcpAccepted	Only required TCP is used.
<Up>_TcpConnected	Only required TCP is used.
<Up>_CopyTxData	Only required if indirect data provision is used. (see TcpIp_UdpTransmit and TcpIp_TcpTransmit)
<Up>_LocalIpAddrAssignmentChg	Optional
<Up>_DhcpEvent	Optional

Table 5-1 Callbacks of TcpIp Socket Ownerss



Note
The prefix “<Up>” is replaced by the name of the TcpIp socket owner.
The API signature of the callbacks is specified by [6] AUTOSAR_SWS_SocketAdaptor.

If the socket owner shall act as a server, a special configuration is needed. On the one hand buffers need to be configured for the number of connections the socket owner wants to handle in parallel. On the other hand we need to configure the parameter ‘Socket Owner Tcp Listen Socket Max’. This value specifies the number of TCP listen sockets this socket

owner will use simultaneously. These pure listen sockets will have no rx or tx buffers. So if a socket owner wants to offer a service using one server socket and accept two parallel connections to its offered service, the parameter ‘Socket Owner Tcp Listen Socket Max’ has to be set to ‘1’ and 2 pairs of rx and tx buffers have to be configured.

5.2.1 <Up>_CopyTxData Callback

If a TcpIpSocketOwner uses one of the APIs TcpIp_UdpTransmit() or TcpIp_TcpTransmit() with indirect data provision (parameter DataPtr == NULL_PTR) the CopyTxData-Callback must be provided by the socket owner.

AUTOSAR specifies that CopyTxData is called one or more times until the socket owner has provided the specified amount of bytes during the Transmit call.

Vector optionally supports an extension of this API using an in/out parameter BufLengthPtr instead of the in-only parameter BufLength. In this case the socket owner may update the value at BufLengthPtr with a lower value in order to indicate that it wants to transmit less data than specified by the Transmit call. Once a socket owner does not use the entire provided buffer in the CopyTxData call, the TcpIp will not request more data, transmit only the provided amount of bytes and release the unused buffer.



Configuration in DaVinci Configurator Pro
The type of the <Up>_CopyTxData callback can be configured individually for each TcpIpSocketOwner using the configuration parameter **TcpIpSocketOwnerCopyTxDataDynamicLengthEnabled**.

5.3 Unicast Address Assignment Methods

The configuration of multiple unicast address assignment methods differs between IPv4 and IPv6.

The reason is that the IPv6 can handle multiple unicast addresses on one IP controller at the same time while the IPv4 only supports one unicast address per IP controller.

Table 5-2 shows the supported configuration possibilities for IPv4 and IPv6:

Configuration Possibilities	IPv4	IPv6
Number of unicast addresses per controller/VLAN	1	1…N
Number of assignment methods per unicast address	1…3	1
Supported address assignment methods	STATIC, LINKLOCAL, DHCPv4	STATIC, LINKLOCAL, DHCPv6, ROUTER
Unicast address assignments that can be stored into nonvolatile memory	STATIC, DHCPv4	STATIC

Table 5-2 Configuration possibilities for IPv4 and IPv6 unicast address assignments

5.3.1 Multiple Address Assignment Methods for IPv4 Unicast Addresses

According to AUTOSAR it is possible to configure up to three different IP address assignment methods for the IPv4 unicast address of an IP instance.

Supported address assignment methods are STATIC, LINKLOCAL (AUTO-IP) and DHCP.

The address assignment methods of a unicast address must be configured with different priorities (1…3, where 1 means highest priority). During runtime the value of the active unicast address is always set to the address of the assignment method with the highest priority that is triggered and ready.

Depending on the configuration parameter TcpIpAssignmentTrigger an address assignment method is either triggered automatically after the ethernet link is up or manually during runtime via the API TcpIp_RequestIpAddrAssignment().

After being triggered the two dynamic address assignment methods DHCP and LINKLOCAL need so time to negotiate an address or even may not be able to configure an address (e.g. because there is no DHCP server present in the network). During this time the address assignment methods are not ready and an address of a method with lower priority will be used if possible.

An address with the STATIC address assignment method can be set in the configuration (see configuration parameter TcpIpStaticIpAddress) or during runtime via TcpIp_RequestIpAddrAssignment().



AUTOSAR Extension
In addition to AUTOSAR an address with assignment method STATIC may be changed even if the assignment trigger is configured to AUTOMATIC. In order to use this feature the configuration parameter TcpIpAssignmentLifetime must be set to TCPIP_STORE.

In this case the configured address value is used as a default value which may be persistently overwritten later. It is also possible to restore the default value again by requesting the address value TCPIP_ADDR_ANY.

All manually triggered address assignment can be deactivated again by calling TcpIp_ReleaseIpAddrAssignment().



AUTOSAR Extension
The AUTOSAR API TcpIp_ReleaseIpAddrAssignment() only supports releasing all manually triggered address assignment methods of an address at once.

Vector provides the additional API TcpIp_ReleaseSpecificIpAddrAssignment() to release only a specific assignment method of an address.

The TcpIp socket owners are notified about each IP address assignment change via the LocalIpAddrAssignmentChg()-Callback.

The following two APIs may be used in context of the callback in order to obtain detailed information about the unassigned/assigned address:

- TcpIp_GetIpAddr
- TcpIp_GetIpAddrCfgSrc

5.3.2 Callout for provision of IPv4 link-local address candidates

The IPv4 link-local address assignment according to IETF RFC 3927 [27] uses a randomly generated address that is most likely unique on the local link. The uniqueness of the address is tested by sending ARP probes for an address candidate before it is assigned as unicast address of the node. If a conflict is detected, another address candidate is generated and tested.

The optional configuration of the callout function <User>_LinkLocalAddrCandidateCallout() allows it to provide address candidates for the link-local address configuration.

Examples of usage scenarios for this callout are:

- Provide link-local address candidates that are derived from likely unique values and therefore reduce the probability of address conflicts.
- Provide stored address candidates that have been tested for uniqueness before and therefore are likely to be still unique on the local-link.

In any case the provided address candidates will be probed for uniqueness again before they are used as source address by the node.

The callout is not required to provide all address candidates. It may also provide only the first candidate. If the callout does not provide an address candidate, the IPv4 will automatically choose a random value according to IETF RFC 3927.



Configuration in DaVinci Configurator Pro
The <User>_LinkLocalAddrCandidateCallout is specified by the optional parameter TcpIpAutoIpAddrCandidateCalloutFunction inside the TcpIpAutoIpConfig container.

5.3.3 Callout for provision of DHCPv4 address that shall be requested

By default the DHCPv4 address assignment does not request the assignment of a particular IP address in the DHCPDISCOVER message.

The optional configuration of the callout function <User>_RequestedDhcpAddrCallout() allows it to provide a particular IP address that shall be requested in the DHCPDISCOVER messages using the ‘Requested IP Address’ option.

This is useful if the client shall try to obtain the same IP address again that was assigned by a DHCP server before.



Configuration in DaVinci Configurator Pro
The <User>_ RequestedDhcpAddrCallout is specified by the optional parameter TcpIpDhcpV4RequestedAddrCalloutFunction inside the TcpIpDhcpConfig container.

5.4 Configuration of Static On-Link Prefixes for IPv6

The IPv6 uses the on-link definition according to the IETF RFCs 4861 [33] and 5942 [41].

If an IPv6 packet shall be sent to a unicast destination the IPv6 determines whether or not the destination is on-link. A destination is on-link if the prefix of the destination address matches `fe80::/10` or any entry in the prefix list.

Packets to on-link destinations are sent directly to the target after the link layer address has been resolved (e.g. via an NDP Neighbor Solicitation).

Packets to destinations that are not known to be on-link must be sent to a router. If there is no router in the network, these IP packets are dropped.

According to IETF RFC 5942 the IPv6 does not treat a prefix derived from a statically or automatically configured address as on-link (except for the link-local prefix `fe80::/10`).

A prefix may be explicitly marked as on-link (added to the prefix list) via a received Router Advertisement containing a “Prefix Information” option with the “L”-flag set or via manual configuration of the prefix.



Manual Configuration of On-Link Prefixes

- The 64 bit prefix of a statically configured IPv6 address can be marked as on-link by setting the configuration parameter TcpIpIpV6AddrPrefixIsOnLink to “Enabled”.
- Alternatively any prefix can by statically added to the prefix list by specifying it via the configuration parameter TcpIpNdpOnLinkPrefix.

5.5 Static and Dynamic Link Layer Address Resolution

Every IPv4 instance has an independent dynamic address resolution cache that can hold a configurable number of entries. Each entry maps an IP address to the corresponding physical (MAC) address.

The entries in the cache are automatically added, updated and replaced by the Address Resolution Protocol (ARP, see [19]) during runtime based on received ARP Request and Reply packets.

In addition to the dynamic address resolution cache a static table can be configured for each IPv4 instance. Static ARP tables may increase performance since no ARP packets need to be sent during runtime. Additionally, some ARP spoofing attacks can be prevented by using static ARP tables because static entries will not be overridden by information from received ARP packets.

A static ARP table may be shared by multiple IPv4 instances.

Each IPv4 instance can use one of the following address resolution modes:

- Dynamic ARP cache only
- Static ARP table only
- Static ARP table and dynamic ARP cache



Configuration in DaVinci Configurator Pro

The size of the dynamic ARP cache is defined by the parameter **[ARP Table Size]** of the **IpV4CtrlConfig**.

Static ARP Tables are defined in an **IpV4StaticArpTable** container that can referenced by the parameter **[Static ARP Table Ref]** of the **IpV4CtrlConfig**.

5.5.1 Prevention of ARP Flooding

Configuration parameter TcpIpArpRequestTimeout is introduced by AUROSAR in order to prevent ARP flooding. If upper layer triggers multiple transmission for the same destination within TcpIpArpRequestTimeout seconds, then only one ARP request will be sent to trigger address resolution. That implies, the ARP Request is sent only for those entries which are not yet valid but exists in Dynamic ARP table.

This feature is disabled, if value of TcpIpArpRequestTimeout parameter is set to 0.

No ARP request is sent, and ARP entry will be discarded from the Dynamic ARP table, once TcpIpArpRetryTime for that ARP entry is expired.



Configuration in DaVinci Configurator Pro

/TcpIp/TcpIpConfig/TcpIpIpConfig/TcpIpIpV4Config/TcpIpArpConfig/TcpIpArpRequestTimeout

/TcpIp/TcpIpConfig/TcpIpIpConfig/TcpIpIpV4Config/TcpIpArpConfig/TcpIpArpRetryTime



Note

If the TcpIp module is configured for TCP, uses the UDP retry queue and/or ICMP Echo Reply functionality, the configuration of TcpIpArpRequestTimeout == 0 may lead to the transmission of ARP request every MainFunction cycle, if the destination address can not be resolved.

5.6 DHCP user options

The DHCPv4 as well as DHCPv6 client implementation of the TcpIp module supports configuration of DHCP user options. These user options are required to:

- Read out the value of received DHCP options by invoking TcpIp_DhcpReadOption() and TcpIp_DhcpV6ReadOption() functions.
- Add custom DHCP options to outgoing DHCP messages by invoking TcpIp_DhcpWriteOption() and TcpIp_DhcpV6WriteOption() functions.

This functionality enables the UpperLayer to include (TX), request (TX) and process (RX) DHCP options, which are not natively supported by the DHCP client implementation.

The configuration of user options shall be done via the configuration parameters listed within the following containers:

> `TcpIp/TcpIpConfig/TcpIpIpConfig/TcpIpIpV4Config/TcpIpDhcpConfig/TcpIpDhcpUserOption`

> `TcpIp/TcpIpConfig/TcpIpIpConfig/TcpIpIpV6Config/TcpIpDhcpV6Config/TcpIpDhcpV6UserOption`

The function <Up_DhcpEvent> (see 4.5.1.10) can be used to synchronize the write and read calls with the DHCP message reception and transmission.

If a DHCP user option is configured with direction `RX_REQUESTED`, the DHCP client will request this option via the Parameter Request List (DHCPv4) or the Option Request Option (DHCPv6).



Example

The functionality of DHCP user options can be uses to handle vendor class and vendor specific information which requires DHCP options as described in:

> DHCPv4: RFC3925: Vendor-Identifying Vendor Options for Dynamic Host Configuration Protocol version 4 (DHCPv4)

> DHCPv6: RFC3315 (chapter 22.16): Dynamic Host Configuration Protocol for IPv6 (DHCPv6)

5.7 IPsec Configuration



Note

The IPsec implementation is limited to the functionality of Authentication Header (AH) processing in combination with IPv4 controllers.
IPv6 as well as Encapsulating Security Payload (ESP) Headers are not supported.

TcpIp allows each controller to optionally reference a `TcpIpIpSecConfigSet` via the `TcpIpIpSecConfigSetRef` which contains the following configurable elements. Each `TcpIpIpSecConfigSet` should specify a tentative number of remote devices that shall use the IPsec communication.

5.7.1 SPD Configuration

The user can configure one or more SPD entries in each `TcpIpIpSecConfigSet` to specify security policies for different types of IP traffic.

While configuring an SPD entry, IP address ranges are configured by START and END parameters, which may be used in three different ways:

> Configure the START value to a valid IP address and do not configure the END value to link this SPD entry to this individual IP address.
> Configure the START as well as the END value to valid IP addresses to link this entry to this range of IP addresses.
> Do not configure START and END values to link this entry will all possible IP address values.



Note
The same configuration variants apply to the configuration of port ranges.

5.7.2 Integrity Algorithms

> The user must configure at least one integrity algorithm to use the IPsec module. The user can select one among the list of supported Integrity Transform Identifiers.

> The user must then configure at least one MAC Generate and Verify CSM job reference each from the CSM module, which matches the identifier selected.



Caution
The CSM module must be configured correctly to ensure functioning of IPsec module. Refer to the CSM technical reference for correct configuration of CSM module.

5.7.3 Encryption Algorithms

This container is not currently supported and must be left empty.

5.8 Static Default Router Configuration

Default router adresses for IpV6 can be configured statically in the Static IPv6 adress container `/TcpIp/TcpIpConfig/TcpIpLocalAddr/TcpIpStaticIpAddressConfig`. If more than one Static IPv6 adress with Default Router is configured for the same controller, any of the default router may be selected during runtime (based on reachability and source address selected). The size of the Default Router List of the NdpConfig must be incremented at least by the number of default routers configured statically. Same entries require only one entry.

5.9 Generation customizations

The TcpIp module uses the ComStackLib to generate the configuration dependent data of the module. The ComStackLib generation behavior can be customized with the parameters of container `/TcpIp/TcpIpGeneral/TcpIpGeneration`. Please refer to the parameter description and to the TechnicalReference_CommonAsr_ComStackLib [7] for more information. Please note that there are customizations which are only valid for a specific quality level (e.g. during the development phase).

6 Security Information

The intention of this chapter is to identify possible configuration-dependent security issues which can arise from using the IP-based protocols provided by the TcpIp module. All mentioned vulnerabilities and possible hardening measures refer to the TcpIp module. However, it is recommended to look at the whole system or network architecture to develop a good security solution. Please note that this list does not thrive for completeness. The user of MICROSAR TCPIP is responsible for the customer-specific configuration of this module.

6.1 ARP

Vulnerability
If `TcpIp/TcpIpGeneral/TcpIpIpV4General/TcpIpArpEnabled` is set to `TRUE` and a dynamic ARP table is configured an attacker could inject forged table entries (known as ARP spoofing).
Attack class
An attacker could perform a man in the middle attack.
Hardening mechanism
Use a static ARP table or a combination of dynamic and static ARP tables instead of only using a dynamic ARP table.

Table 6-1 TcpIpArpEnabled, ARP spoofing

Vulnerability
If `TcpIp/TcpIpGeneral/TcpIpIpV4General/TcpIpArpEnabled` is set to `TRUE` and a dynamic ARP table is used an attacker could flood the table with forged table entries (known as ARP flooding).
Attack class
This could lead to time lags on the communication channel.
Hardening mechanism
Use a static ARP table or a combination of dynamic and static ARP tables instead of only using a dynamic ARP table.

Table 6-2 TcpIpArpEnabled, ARP flooding

Vulnerability
If `TcpIp/TcpIpGeneral/TcpIpIpV4General/TcpIpArpDiscardedEntryHandling` is set to `TRUE` new ARP responses are discarded if the ARP table is full.
Attack class
An attacker could fill the ARP table with forged table entries and block the table for valid incoming ARP responses, which leads to a denial of service.
Hardening mechanism
Use a static ARP table or a combination of dynamic static ARP tables instead of only using a dynamic ARP table. Configure a reasonable timeout for the ARP entries in a dynamic ARP table.

Table 6-3 TcpIpArpDiscardedEntryHandling

6.2 ICMPv4

Vulnerability
If `TcpIp/TcpIpGeneral/TcpIpIpV4General/TcpIpIcmpEnabled` is set to `TRUE` the TcpIp stack can process and send ICMP messages (e.g. echo response).
Attack class
ICMP messages could reveal useful information to an attacker e.g. with an ICMP echo request an attacker could enumerate which hosts are available in a network.
Hardening mechanism
Use an additional system like a firewall to block certain packets or an intrusion detection system to detect anomalies. In addition, it is possible to deactivate ICMPv4 if not necessary.

Table 6-4 TcpIpIcmpEnabled

6.3 IPv4

Vulnerability
If `TcpIp/TcpIpGeneral/TcpIpIpV4General/TcpIpIpV4Enabled` is set to `TRUE` IP spoofing is possible. That means an attacker could pretend to be someone else.
Attack class
The authenticity of IP packets could be threatened.
Hardening mechanism
An additional security protocol (e.g. Internet Protocol Security (IPsec)) can ensure the authenticity of IP packets. Furthermore, a well-grounded network architecture (e.g. use virtual LANs) and the use of a firewall or an intrusion detection system can harden the network.

Table 6-5 TcpIpIpV4Enabled

6.4 IPv6

Vulnerability
In case `TcpIp/TcpIpGeneral/TcpIpIpV6General/TcpIpIpV6Enabled` is set to `TRUE` IP spoofing is possible. That means an attacker could pretend to be someone else.

Attack class
The authenticity of IP packets could be threatened.
Hardening mechanism
An additional security protocol (e.g. Internet Protocol Security (IPsec)) can ensure the authenticity of IP packets. Furthermore, a well-grounded network architecture (e.g. use virtual LANs) and the use of a firewall or an intrusion detection system can harden the network.

Table 6-6 TcpIpIpV6Enabled

6.5 TCP

Vulnerability
In case `TcpIp/TcpIpGeneral/TcpIpTcpEnabled` is set to `TRUE` an attacker could use port scan techniques based on the TCP flags.
Attack class
This port scanning techniques could reveal useful information about open ports on a system.
Hardening mechanism
The use of a firewall could make port scanning more difficult.

Table 6-7 TcpIpTcpEnabled, port scan

Vulnerability
If `TcpIp/TcpIpGeneral/TcpIpTcpEnabled` is set to `TRUE` an attacker could read and manipulate the communication between two hosts.
Attack class
An attacker could perform a man in the middle attack.
Hardening mechanism
Use an additional security protocol (e.g. Transport Layer Security (TLS)) for authentication and data encryption.

Table 6-8 TcpIpTcpEnabled, man in the middle

6.6 UDP

Vulnerability
If `TcpIp/TcpIpGeneral/TcpIpUdpEnabled` is set to `TRUE` an attacker could read and manipulate the communication between two hosts.
Attack class
An attacker could perform a man in the middle attack.
Hardening mechanism
Use an additional security protocol (e.g. Internet Protocol Security (IPsec)) for authentication and data encryption.

Table 6-9 TcpIpUdpEnabled

6.7 DHCPv4 Server

Vulnerability
If `TcpIp/TcpIpGeneral/TcpIpDhcpServerEnabled` is set to `TRUE` an attacker could consume all available IP addresses in the configured IP range (known as DHCP starvation attack).
Attack class
If all IP addresses were consumed by an attacker a new client is not able to receive an IP address, which leads to a denial of service.
Hardening mechanism
An intrusion detection system could be used to detect anomalies.

Table 6-10 TcpIpDhcpServerEnabled

7 Glossary and Abbreviations

7.1 Glossary

Term	Description

Table 7-1 Glossary

7.2 Abbreviations

Abbreviation	Description
AH	Authentication Header
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
CSM	Crypto Services Module
DEM	Diagnostic Event Manager
DET	Development Error Tracer
EAD	Embedded Architecture Designer
ECU	Electronic Control Unit
ESP	Encapsulated Security Payload
ISR	Interrupt Service Routine
IPsec	Internet Protocol Security
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
SA	Security Association
SAD	Security Association Database
SPD	Security Policy Database
SRS	Software Requirement Specification
SWS	Software Specification

Table 7-2 Abbreviations

8 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com